
GIPSI

Plataforma de Gestión de Citas para Negocios de Estética y Bienestar

Documentación Técnica del Proyecto

Trabajo de Fin de Ciclo / Proyecto de Desarrollo de Software

Autores: José Ramón López Guisado y Adrián Linares Utrera

Repositorio Backend: github.com/Joserraa05/Gipsi-API

Repositorio Frontend: github.com/Joserraa05/Gipsi-Frontend

Tecnologías: Spring Boot 3.4 · Flutter 3.24 · PostgreSQL

Fecha: 2026-05-13

Resumen

Gipsi es una aplicación móvil multiplataforma orientada al sector de la estética, peluquería y bienestar personal. Su propósito principal es digitalizar y simplificar la gestión de citas entre clientes y negocios, permitiendo que los usuarios puedan consultar servicios, reservar horarios disponibles, recibir notificaciones y valorar su experiencia, mientras que los negocios pueden gestionar su disponibilidad, trabajadores, servicios y agenda desde una solución centralizada.

El sistema se compone de dos grandes bloques tecnológicos. Por un lado, una API REST desarrollada con Spring Boot 3.4 sobre Java 21, encargada de exponer los servicios de negocio, gestionar la autenticación, aplicar las reglas de seguridad, persistir la información y coordinar la lógica principal de la aplicación. Por otro lado, un frontend móvil construido con Flutter 3.24 en Dart, organizado como monorepo mediante Melos, lo que permite dividir el proyecto en varios paquetes reutilizables y mantener una estructura modular, escalable y fácil de mantener.

El backend implementa autenticación mediante JSON Web Tokens, utilizando access tokens de corta duración y refresh tokens de larga duración con rotación en cada uso. Este mecanismo permite mantener sesiones seguras sin obligar al usuario a iniciar sesión continuamente. Además, incorpora control de acceso basado en roles, diferenciando permisos y funcionalidades según el tipo de usuario: cliente, negocio o trabajador.

La aplicación también incluye gestión de disponibilidad por franjas horarias, permitiendo calcular huecos libres en función de los horarios del negocio, la duración de los servicios y las reservas ya existentes. A esto se suma un sistema de valoraciones con agregados calculados dinámicamente, notificaciones push mediante Firebase Cloud Messaging y almacenamiento de imágenes mediante una capa de abstracción preparada para migrar en el futuro a servicios compatibles con S3, como Amazon S3 o soluciones equivalentes de almacenamiento de objetos.

La base de datos relacional utilizada es PostgreSQL. Su estructura se gestiona mediante migraciones versionadas con Flyway, lo que permite controlar la evolución del esquema de forma ordenada, reproducible y segura entre distintos entornos. La documentación de la API se genera automáticamente con Springdoc OpenAPI, facilitando la consulta de endpoints, parámetros, respuestas y modelos de datos tanto para desarrolladores como para futuras integraciones externas.

El frontend adopta una arquitectura en capas basada en el patrón Repository. Esta separación permite aislar la lógica de acceso a datos de la interfaz de usuario y facilita el mantenimiento, las pruebas y la evolución del código. Para la gestión de estado se utiliza Riverpod, una solución reactiva que permite conectar la interfaz con los datos de forma controlada y predecible. La navegación se gestiona con `go_router`, que permite

definir rutas de forma declarativa, y la comunicación HTTP se realiza mediante Dio, incluyendo interceptores para automatizar tareas como la inserción del token de acceso o el refresco automático de credenciales.

El sistema de diseño propio, denominado `gipsi_shared_ui`, centraliza tokens de color, estilos tipográficos, espaciados y componentes reutilizables. Gracias a esta capa compartida, la aplicación mantiene una apariencia coherente entre pantallas y reduce la duplicación de código visual.

Este documento recoge el análisis de requisitos, las decisiones de diseño arquitectónico, los detalles de implementación, las estrategias de prueba y las instrucciones de despliegue, con el objetivo de servir como referencia técnica completa del proyecto Gipsi.

Palabras clave: aplicación móvil, Flutter, Spring Boot, REST API, JWT, PostgreSQL, gestión de citas, Riverpod, Melos, Firebase.

Índice general

Resumen	2
Índice de figuras	6
Índice de tablas	7
1 Introducción y Contexto	11
1.1 Motivación del Proyecto	11
1.2 Descripción General del Sistema	11
1.3 Alcance de la Versión Actual	12
1.4 Objetivos del Proyecto	13
1.5 Estructura del Documento	13
2 Análisis y Requisitos	15
2.1 Identificación de Actores	15
2.2 Requisitos Funcionales	15
2.2.1 Módulo de Autenticación (RF-1)	15
2.2.2 Módulo de Negocios (RF-2)	16
2.2.3 Módulo de Servicios (RF-3)	17
2.2.4 Módulo de Reservas (RF-4)	18
2.2.5 Módulo de Valoraciones (RF-5)	19
2.2.6 Módulo de Equipo y Horarios (RF-6)	19
2.2.7 Módulo de Perfil y Favoritos (RF-7)	20
2.3 Requisitos No Funcionales	21
2.4 Casos de Uso Principales	25
2.4.1 CU-01: Registro de Cliente	25
2.4.2 CU-02: Exploración y Descubrimiento de Negocios	25
2.4.3 CU-03: Consulta de Disponibilidad	26
2.4.4 CU-04: Dejar Valoración	26
2.4.5 CU-05: Crear Reserva	27
2.4.6 CU-06: Gestionar Favoritos	27
2.4.7 CU-07: Cancelar Reserva	28
2.4.8 CU-08: Vincular Trabajador a un Negocio	28
2.4.9 CU-09: Gestionar Ausencias del Equipo	29
2.5 Modelo de Dominio Conceptual	29
3 Diseño	33
3.1 Arquitectura General del Sistema	33
3.1.1 Principios Arquitectónicos Aplicados	33

3.2	Arquitectura del Backend (Spring Boot)	34
3.2.1	Estructura de Paquetes	34
3.2.2	Capa de Seguridad: JWT y Spring Security	35
3.2.3	Cálculo de Disponibilidad Horaria	37
3.3	Arquitectura del Frontend (Flutter)	38
3.3.1	Monorepo con Melos	38
3.3.2	Gestión de Estado con Riverpod	38
3.3.3	Navegación con go_router	39
3.3.4	Sistema de Diseño: gipsi_shared_ui	40
3.4	Modelo de Datos Relacional	40
3.4.1	Esquema Principal	40
3.4.2	Gestión de Migraciones con Flyway	41
3.5	Diseño de la API REST	42
3.5.1	Convenciones Generales	42
3.5.2	Catálogo de Endpoints	42
4	Implementación	44
4.1	Backend: Detalles de Implementación	44
4.1.1	Configuración de Spring Security	44
4.1.2	Rate Limiting con Bucket4j	45
4.1.3	Notificaciones Push con Firebase	45
4.1.4	Jobs Programados	46
4.2	Frontend: Detalles de Implementación	46
4.2.1	Cliente HTTP con Dio e Interceptor de Refresh	46
4.2.2	Pantalla de Exploración	47
4.2.3	Pantalla de Detalle de Negocio	47
4.2.4	Almacenamiento Seguro de Tokens	48
5	Pruebas y Evaluación	49
5.1	Estrategia de Pruebas	49
5.2	Pruebas Funcionales	50
5.2.1	Pruebas del Flujo de Autenticación	50
5.2.2	Pruebas del Módulo de Exploración (Frontend)	51
5.2.3	Pruebas del Módulo de Disponibilidad	51
5.3	Pruebas Unitarias	51
5.3.1	Pruebas del Backend con JUnit 5	51
5.4	Evaluación del Sistema	57
5.4.1	Pruebas de Rendimiento Manual	57
5.5	Capturas de las Pruebas	57
5.5.1	Pruebas del flujo de autenticación	57
5.5.2	Pruebas del módulo de exploración	62
5.5.3	Pruebas del módulo de disponibilidad	66
5.5.4	Capturas de pruebas unitarias del backend	69
6	Despliegue	72
6.1	Requisitos de Entorno	72
6.1.1	Backend	72
6.1.2	Frontend	73
6.1.3	Servicios Externos y Red	73

6.2	Configuración del Backend	73
6.2.1	Variables de Entorno	73
6.2.2	Proceso de Arranque	74
6.3	Configuración del Frontend	75
6.3.1	Bootstrap del Monorepo	75
6.3.2	Compilación para Distribución	76
6.4	Despliegue con Docker (Recomendado para Producción)	76
6.4.1	Archivo de Variables para Docker	77
6.4.2	Secuencia de Puesta en Producción	78
6.4.3	Verificaciones Posteriores y Mantenimiento	78
7	Conclusiones y Posibles Mejoras	79
7.1	Conclusiones	79
7.1.1	Logros Técnicos Destacados	79
7.1.2	Lecciones Aprendidas	79
7.2	Posibles Mejoras	80
7.2.1	Mejoras a Corto Plazo (Bloques Pendientes)	80
7.2.2	Mejoras Arquitectónicas	80
7.2.3	Mejoras de Seguridad	81
	Bibliografía	82
A	Configuración Completa del pom.xml	84
B	Configuración del Monorepo (melos.yaml)	87
C	Usuarios de Prueba (DataInitializer)	88
D	Formato de Respuesta de Disponibilidad	89
E	Glosario de Términos	91
E.1	Glosario de términos técnicos	91

Índice de figuras

2.1	Vista núcleo del modelo de dominio de Gipsi	30
2.2	Vista operativa y de soporte del modelo de dominio	31
3.1	Arquitectura general del sistema Gipsi	33
3.2	Flujo de autenticación en el backend	36
5.1	Pirámide de testing aplicada a Gipsi	49
5.2	PF-A01 — Login con credenciales correctas	59
5.3	PF-A02 — Login con contraseña incorrecta	59
5.4	PF-A03 — Rate limiting tras 11 intentos de login	60

5.5	PF-A04 — Refresh con token válido	60
5.6	PF-A05 — Refresh con token ya rotado	61
5.7	PF-A06 — Acceso al endpoint /me sin token	61
5.8	PF-A07 — Acceso a recurso de negocio con rol CUSTOMER	62
5.9	PF-A08 — Registro con email ya existente	62
5.10	PF-E01 — Carga inicial de la pestaña Explorar	64
5.11	PF-E02 — Filtrado por categoría Peluquería	64
5.12	PF-E03 — Recarga manual con pull-to-refresh	65
5.13	PF-E04 — Navegación al detalle de negocio	65
5.14	PF-E05 — Estado de error con backend caído	66
5.15	PF-E06 — Estado vacío sin resultados	66
5.16	PF-D01 — Disponibilidad en día laborable	67
5.17	PF-D02 — Disponibilidad fuera de horario	68
5.18	PF-D03 — Slot marcado como BOOKED	68
5.19	PF-D04 — Disponibilidad agregada en negocio multi-worker	69
5.20	PU-B01 — Ejecucion completa de JwtServiceTest	70
5.21	PU-B02 — Ejecucion completa de AvailabilityServiceTest	71

Índice de tablas

2.1	Actores del sistema Gipsi	15
2.2	Requisitos funcionales — Módulo de Autenticación	16
2.3	Requisitos funcionales — Módulo de Negocios	16
2.4	Requisitos funcionales — Módulo de Servicios	17
2.5	Requisitos funcionales — Módulo de Reservas	18
2.6	Requisitos funcionales — Módulo de Valoraciones	19
2.7	Requisitos funcionales — Módulo de Equipo y Horarios	20
2.8	Requisitos funcionales — Módulo de Perfil y Favoritos	20
2.9	Requisitos no funcionales	21
3.1	Tokens de color del sistema de diseño Gipsi	40
3.2	Endpoints de la API — Autenticación y Perfil	42
3.3	Endpoints de la API — Negocios y Servicios	42
3.4	Endpoints de la API — Reservas y Valoraciones	43
5.1	Casos de prueba — Autenticación	50
5.2	Casos de prueba — Exploración en frontend	51
5.3	Casos de prueba — Disponibilidad horaria	51
5.4	Tiempos de respuesta medidos en entorno local	57
5.5	Guía de ejecución manual para capturas del flujo de autenticación	58
5.6	Guía de ejecución manual para capturas del módulo de exploración	63
5.7	Guía de ejecución manual para capturas del módulo de disponibilidad	67

5.8	Resumen del procedimiento seguido para las capturas de las pruebas unitarias del backend	70
6.1	Requisitos de entorno del backend	72
6.2	Requisitos de entorno del frontend	73
6.3	Servicios externos necesarios para producción	73
C.1	Usuarios de prueba creados por DataInitializer	88

Listings

3.1	Estructura de paquetes del backend	34
3.2	Fragmento conceptual del cálculo de disponibilidad	37
3.3	Estructura del monorepo Flutter	38
3.4	Esquema simplificado de rutas con go_router	39
3.5	DDL simplificado de las tablas principales (Flyway V1)	40
4.1	Configuración de Spring Security (SecurityConfig.java)	44
4.2	Envío asíncrono de notificación push	45
4.3	Job de auto-cancelación con @Scheduled	46
4.4	Interceptor de refresh token en Dio	47
4.5	Servicio de almacenamiento seguro de tokens	48
5.1	Pruebas unitarias de JwtService	52
5.2	Pruebas unitarias del servicio de disponibilidad	53
6.1	Variables de entorno del backend	73
6.2	Comandos de arranque del backend	74
6.3	Preparación recomendada de PostgreSQL	75
6.4	Proceso de setup del frontend	75
6.5	Compilación para Android e iOS	76
6.6	docker-compose.yml para producción	76
6.7	Ejemplo de fichero .env de producción	77
6.8	Secuencia básica de despliegue Docker	78
6.9	Copia de seguridad básica de PostgreSQL	78
A.1	pom.xml del proyecto Gipsi-API	84
B.1	melos.yaml del monorepo Flutter	87
D.1	Ejemplo de respuesta del endpoint de disponibilidad	89

Capítulo 1

Introducción y Contexto

1.1. Motivación del Proyecto

El sector de la estética, la peluquería y el bienestar personal mueve en España miles de millones de euros anuales y cuenta con cientos de miles de pequeños negocios distribuidos por todo el territorio. A pesar de su relevancia económica, la mayor parte de estos establecimientos sigue gestionando sus citas de forma analógica (por teléfono, agenda física o, como mucho, mediante grupos de mensajería) con los problemas derivados que esto conlleva: dobles reservas, olvidos, dificultad para mostrar disponibilidad en tiempo real y ausencia de un canal digital propio para el cliente.

La proliferación de teléfonos inteligentes ha transformado el comportamiento del consumidor: hoy el usuario espera poder reservar un servicio a cualquier hora, desde cualquier lugar y de forma instantánea, de la misma manera en que reserva un hotel o pide comida a domicilio. Esta brecha entre las expectativas del cliente digital y la realidad operativa del pequeño negocio de belleza es la motivación principal de Gipsi.

Asimismo, el análisis del mercado actual muestra que muchas de las soluciones existentes presentan una relación calidad-precio poco equilibrada: ofrecen funcionalidades limitadas, interfaces poco cuidadas o una experiencia de usuario mejorable, mientras mantienen costes relativamente elevados para pequeños negocios. Apoyándose en la teoría de los océanos azules, Gipsi identifica una oportunidad para alejarse de la competencia directa basada únicamente en precio o presencia en el mercado, y propone cubrir un hueco poco explotado: una aplicación accesible económicamente, sencilla de adoptar por negocios pequeños y medianos, pero con un nivel de calidad técnica, visual y funcional superior al habitual en este segmento. De este modo, el proyecto busca crear un espacio de valor diferenciado, donde el cliente profesional no tenga que elegir entre una herramienta barata pero limitada o una plataforma completa pero difícil de asumir económicamente.

1.2. Descripción General del Sistema

Gipsi es una plataforma de gestión de citas dirigida al sector de la estética y el bienestar. Su propósito es doble:

- Para el **cliente**: descubrir negocios cercanos por categoría (peluquería, nail art, spa, barbería, estética...), consultar servicios y precios, ver disponibilidad en

tiempo real y gestionar sus reservas desde el móvil.

- Para el **negocio**: disponer de una agenda digital que elimine la necesidad de gestionar citas por teléfono, recibir notificaciones push ante nuevas reservas y cancelaciones, y obtener visibilidad ante clientes potenciales en su área.

El sistema distingue tres tipos de actores: **clientes**, **negocios** (propietarios) y **trabajadores** (empleados de un negocio), cada uno con permisos y vistas diferenciadas.

1.3. Alcance de la Versión Actual

La versión documentada en este proyecto, denominada internamente *v2*, recoge tanto las funcionalidades ya implementadas en el repositorio como aquellas que se encuentran en fase de integración y que estarán disponibles para la presentación del proyecto. El objetivo de esta versión es mostrar una experiencia funcional completa del producto, desde el descubrimiento de negocios hasta la reserva y gestión básica de citas.

1. Registro y autenticación de usuarios con gestión de sesiones mediante tokens JWT, refresh tokens y refresco automático de credenciales.
2. Control de acceso basado en roles, diferenciando entre usuarios cliente, negocios y trabajadores, de forma que cada perfil acceda únicamente a las funcionalidades correspondientes.
3. Exploración de negocios con paginación, búsqueda por texto, filtro por categoría y ciudad.
4. Vista de detalle de negocio con portada colapsable, información de contacto, catálogo de servicios con precios y duraciones, trabajadores disponibles y sección de valoraciones.
5. Flujo completo de reserva dividido en varios pasos: selección de servicio, selección de trabajador, elección de fecha y hora disponible, revisión de los datos y confirmación final de la cita.
6. Cálculo de disponibilidad horaria por servicio y trabajador, teniendo en cuenta la duración del servicio, los horarios configurados y las reservas ya existentes.
7. Gestión de citas propias por parte del cliente, permitiendo consultar reservas futuras, revisar el histórico de citas y acceder al detalle de cada reserva.
8. Sistema de reviews sobre reservas confirmadas y finalizadas, evitando que se puedan valorar servicios no realizados.
9. Sistema de favoritos para que los usuarios puedan guardar negocios de interés y acceder posteriormente a ellos de forma rápida.
10. Notificaciones push mediante Firebase Cloud Messaging para informar al usuario sobre eventos relevantes, como confirmaciones, recordatorios o cambios relacionados con sus citas.
11. Gestión de imágenes de negocios y servicios mediante una capa de almacenamiento local abstraída, preparada para una futura migración a servicios compatibles con S3.
12. Rate limiting en los endpoints de autenticación, con el objetivo de proteger la

API frente a abuso, ataques automatizados o intentos repetidos de acceso.

13. Panel de gestión para negocios, desde el que la empresa podrá administrar su información pública, servicios, trabajadores, horarios y citas recibidas.
14. Búsqueda avanzada con filtros adicionales, orientada a mejorar el descubrimiento de negocios según criterios como categoría, ubicación, disponibilidad o valoración.

Como líneas de evolución futura, se plantea ampliar la vista de empresa con funcionalidades de gestión más avanzadas. Entre ellas destaca la incorporación de un sistema CRM que permita a los negocios organizar su relación con los clientes, consultar historiales de citas, segmentar usuarios y mejorar la comunicación comercial. Asimismo, se contempla la integración de un módulo de facturación que facilite la emisión y consulta de facturas, el control de servicios cobrados y la gestión económica básica del negocio desde la propia plataforma.

Estas mejoras permitirían que Gipsi evolucionara desde una aplicación de reserva de citas hacia una solución integral de gestión para pequeños y medianos negocios del sector de la estética, la peluquería y el bienestar personal.

1.4. Objetivos del Proyecto

Los objetivos técnicos que guían el desarrollo son:

Objetivos Técnicos

- OT1.** Diseñar e implementar una API REST segura, versionable y bien documentada.
- OT2.** Construir una aplicación móvil multiplataforma (Android e iOS) con una única base de código.
- OT3.** Garantizar la seguridad del sistema mediante JWT con rotación de refresh tokens siguiendo las recomendaciones OWASP.
- OT4.** Establecer una arquitectura escalable que permita la migración futura a infraestructura cloud (S3, servicios gestionados).
- OT5.** Organizar el frontend como monorepo con separación clara de responsabilidades entre packages.
- OT6.** Documentar la API de forma automática y ejecutable desde el propio servidor.

1.5. Estructura del Documento

El presente documento se organiza en los siguientes capítulos:

Capítulo 2 — Análisis y Requisitos: Especificación de requisitos funcionales y no funcionales, casos de uso y modelo de dominio.

Capítulo 3 — Diseño: Arquitectura del sistema, modelo de datos, diseño de la API y sistema de diseño visual.

Capítulo 4 — Implementación: Detalle técnico de los componentes clave del backend y el frontend.

Capítulo 5 — Pruebas y Evaluación: Estrategia de pruebas, pruebas funcionales

y unitarias.

Capítulo 6 — Despliegue: Requisitos, configuración de entorno y proceso de puesta en producción.

Capítulo 7 — Conclusiones y Mejoras: Valoración del trabajo realizado y líneas de trabajo futuro.

Bibliografía y Anexos: Referencias técnicas y material complementario.

Capítulo 2

Análisis y Requisitos

2.1. Identificación de Actores

El sistema identifica tres tipos de actores primarios y un actor secundario:

Tabla 2.1: Actores del sistema Gipsi

Actor	Descripción
Cliente (CUSTOMER)	Usuario final que descubre negocios, explora servicios y gestiona sus reservas. Es el actor más numeroso del sistema.
Negocio (BUSINESS)	Propietario o gestor de un establecimiento. Define los servicios, horarios y asigna trabajadores. Recibe notificaciones de nuevas reservas.
Trabajador (WORKER)	Empleado de un negocio que ejecuta los servicios. Tiene su propio horario y días libres, lo que afecta directamente a la disponibilidad calculada.
Sistema (actor secundario)	Ejecuta tareas programadas: cancelación automática de reservas pendientes antiguas y envío de recordatorios diarios.

2.2. Requisitos Funcionales

Los requisitos funcionales se clasifican por módulo. En la notación utilizada, **RF** identifica requisito funcional, el primer número indica el módulo y el segundo el requisito dentro del módulo.

2.2.1. Módulo de Autenticación (RF-1)

Tabla 2.2: Requisitos funcionales — Módulo de Autenticación

ID	Descripción	Prioridad
RF-1.1	El sistema debe permitir el registro de nuevos usuarios (clientes) con nombre, email, nombre de usuario, teléfono y contraseña.	Alta
RF-1.2	El sistema debe autenticar usuarios mediante usuario/contraseña y devolver un <i>access token</i> (30 min) y un <i>refresh token</i> (30 días).	Alta
RF-1.3	El sistema debe permitir la renovación del <i>access token</i> presentando un <i>refresh token</i> válido. La rotación es obligatoria: se emite un nuevo par de tokens y el anterior queda invalidado.	Alta
RF-1.4	El sistema debe permitir el cierre de sesión explícito revocando el <i>refresh token</i> .	Alta
RF-1.5	El sistema debe limitar a 10 peticiones por minuto y por dirección IP los intentos de login y registro para mitigar ataques de fuerza bruta.	Alta
RF-1.6	El endpoint <code>GET /me</code> debe devolver el perfil completo del usuario autenticado con los campos específicos de su rol.	Media
RF-1.7	El sistema debe permitir el registro de negocios con nombre, email, nombre de usuario, teléfono, contraseña, CIF, dirección, ciudad, descripción y modo autónomo.	Alta
RF-1.8	El sistema debe permitir el registro de trabajadores y su vinculación posterior a un negocio mediante invitación o código temporal.	Alta
RF-1.9	El CIF de cada negocio debe ser único en el sistema.	Alta
RF-1.10	El sistema debe validar el formato de correo electrónico en los procesos de registro y actualización de perfil.	Alta
RF-1.11	Los usuarios autenticados deben poder cambiar su contraseña y revocar todas sus sesiones activas.	Media

2.2.2. Módulo de Negocios (RF-2)

Tabla 2.3: Requisitos funcionales — Módulo de Negocios

ID	Descripción	Prioridad
RF-2.1	El sistema debe permitir listar negocios con paginación, búsqueda por texto libre (<i>q</i>), filtro por ciudad y filtro por categoría.	Alta

Continúa en la siguiente página

Continuación de la tabla anterior

ID	Descripción	Prioridad
RF-2.2	El sistema debe mostrar el detalle de un negocio, incluyendo sus servicios, agregados de valoraciones y datos de contacto.	Alta
RF-2.3	Un negocio debe poder subir y gestionar imágenes de perfil y portada.	Media
RF-2.4	El sistema debe soportar el concepto de negocio autónomo (un solo trabajador), lo que simplifica la selección de trabajador en la reserva.	Media
RF-2.5	Los agregados de rating (media y número de valoraciones) deben estar disponibles en la respuesta de negocio sin necesidad de consultas adicionales.	Media
RF-2.6	Un negocio autenticado debe poder consultar y editar su información pública, incluyendo nombre, email, teléfono, dirección, ciudad y descripción.	Alta
RF-2.7	El negocio debe poder configurar parámetros operativos como modo autónomo, autoaceptación de reservas, horizonte de disponibilidad, umbral de autocancelación de reservas pendientes y activación del fichaje.	Media
RF-2.8	El sistema debe exponer los listados públicos de trabajadores y servicios asociados a cada negocio.	Media
RF-2.9	El sistema debe reflejar si un negocio ha completado su configuración mínima cuando disponga de al menos un servicio y un horario operativo.	Media

2.2.3. Módulo de Servicios (RF-3)

Tabla 2.4: Requisitos funcionales — Módulo de Servicios

ID	Descripción	Prioridad
RF-3.1	Cada servicio debe tener asociado un nombre, duración en minutos, precio y opcionalmente una imagen.	Alta
RF-3.2	El sistema debe calcular la disponibilidad de un servicio para una fecha dada, devolviendo una lista de slots horarios con indicación de disponibilidad y causa de indisponibilidad.	Alta
RF-3.3	La disponibilidad debe considerar el horario del negocio, las reservas existentes, los días libres del trabajador y si se permite trabajo en horas extra.	Alta

Continúa en la siguiente página

Continuación de la tabla anterior

ID	Descripción	Prioridad
RF-3.4	Los servicios deben poder filtrarse por texto, ciudad y categoría.	Media
RF-3.5	Un negocio autenticado debe poder crear, editar y eliminar sus servicios.	Alta
RF-3.6	Un negocio debe poder activar o desactivar temporalmente un servicio sin eliminarlo.	Alta
RF-3.7	En negocios no autónomos, la empresa debe poder asignar y desasignar trabajadores a cada servicio.	Alta
RF-3.8	La duración de un servicio debe ser positiva y múltiplo de 15 minutos.	Alta
RF-3.9	Solo los servicios activos deben poder reservarse.	Alta

2.2.4. Módulo de Reservas (RF-4)

Tabla 2.5: Requisitos funcionales — Módulo de Reservas

ID	Descripción	Prioridad
RF-4.1	Un cliente autenticado debe poder crear una reserva para un servicio en un slot de tiempo disponible.	Alta
RF-4.2	El sistema debe gestionar los estados de una reserva: PENDING , CONFIRMED , CANCELED , COMPLETED .	Alta
RF-4.3	El sistema debe cancelar automáticamente mediante un job programado las reservas en estado PENDING que superen un umbral de tiempo sin confirmación.	Media
RF-4.4	El sistema debe enviar recordatorios push el día anterior a una reserva confirmada.	Media
RF-4.5	Un cliente debe poder cancelar sus propias reservas dentro de un plazo definido.	Alta
RF-4.6	Solo los clientes autenticados deben poder crear reservas.	Alta
RF-4.7	La creación de una reserva debe validar que la fecha solicitada esté alineada a slots de 15 minutos y no supere el horizonte de disponibilidad del negocio.	Alta
RF-4.8	En negocios no autónomos, cada reserva debe asociarse a un trabajador perteneciente al negocio, disponible y habilitado para prestar el servicio seleccionado.	Alta

Continúa en la siguiente página

Continuación de la tabla anterior

ID	Descripción	Prioridad
RF-4.9	El sistema debe permitir que trabajadores y negocios consulten las reservas que tienen asignadas o recibidas según su rol.	Alta
RF-4.10	El negocio o el trabajador asignado deben poder confirmar reservas pendientes.	Alta
RF-4.11	El cliente propietario, el negocio o el trabajador asignado deben poder cancelar una reserva, opcionalmente indicando el motivo de cancelación.	Alta
RF-4.12	El estado inicial de una reserva debe ser CONFIRMED cuando el negocio o el trabajador tengan autoaceptación habilitada y PENDING en caso contrario.	Media
RF-4.13	El sistema debe enviar notificaciones push en eventos de creación, confirmación, cancelación y recordatorio de reservas.	Media

2.2.5. Módulo de Valoraciones (RF-5)

Tabla 2.6: Requisitos funcionales — Módulo de Valoraciones

ID	Descripción	Prioridad
RF-5.1	Un cliente solo puede dejar una valoración sobre una reserva en estado CONFIRMED cuya fecha de fin haya pasado.	Alta
RF-5.2	Solo puede existir una valoración por reserva (restricción UNIQUE en base de datos).	Alta
RF-5.3	El cliente propietario de la valoración puede editarla o eliminarla. Los negocios no pueden eliminar valoraciones.	Alta
RF-5.4	Las valoraciones deben poder consultarse públicamente paginadas por negocio o por servicio.	Alta
RF-5.5	Los agregados (media de rating, número de valoraciones) se calculan dinámicamente y se incluyen en las respuestas de negocio y servicio.	Media
RF-5.6	El sistema debe notificar al negocio cuando reciba una nueva valoración.	Media

2.2.6. Módulo de Equipo y Horarios (RF-6)

Tabla 2.7: Requisitos funcionales — Módulo de Equipo y Horarios

ID	Descripción	Prioridad
RF-6.1	Un negocio debe poder generar y revocar invitaciones temporales para incorporar trabajadores a su equipo.	Alta
RF-6.2	Un trabajador debe poder aceptar o canjear una invitación válida para vincularse a un negocio.	Alta
RF-6.3	El negocio debe poder consultar su equipo, modificar ciertos ajustes del trabajador y desvincularlo sin eliminar su historial.	Media
RF-6.4	El negocio autónomo y los trabajadores deben poder definir y actualizar su horario semanal de trabajo.	Alta
RF-6.5	El negocio debe poder consultar y modificar el horario semanal de sus trabajadores.	Alta
RF-6.6	El negocio autónomo o el trabajador deben poder registrar periodos de indisponibilidad que bloqueen la reserva en ese rango.	Media
RF-6.7	Los trabajadores deben poder solicitar ausencias y el negocio debe poder aprobarlas, rechazarlas o registrar ausencias directas.	Media
RF-6.8	Antes de aprobar o registrar una ausencia, el sistema debe permitir consultar las reservas activas que entren en conflicto con ese periodo.	Media
RF-6.9	Si el fichaje está habilitado, el trabajador debe poder realizar entrada, salida y consultar su sesión abierta actual.	Baja

2.2.7. Módulo de Perfil y Favoritos (RF-7)

Tabla 2.8: Requisitos funcionales — Módulo de Perfil y Favoritos

ID	Descripción	Prioridad
RF-7.1	El cliente debe poder consultar y editar su perfil, incluyendo su imagen de avatar.	Media
RF-7.2	El cliente debe poder eliminar su cuenta, el sistema debe cancelar sus reservas futuras activas y anonimizar sus datos personales conservando la integridad del historial.	Media
RF-7.3	Los clientes deben poder marcar y desmarcar negocios como favoritos.	Media
RF-7.4	Los clientes deben poder marcar y desmarcar servicios como favoritos.	Media

Continúa en la siguiente página

Continuación de la tabla anterior

ID	Descripción	Prioridad
RF-7.5	Los usuarios autenticados deben poder registrar y eliminar dispositivos para recibir notificaciones push.	Media

2.3. Requisitos No Funcionales

Tabla 2.9: Requisitos no funcionales

ID	Descripción	Categoría
RNF-1	La API debe responder en menos de 500 ms bajo carga normal para endpoints de consulta sin agregaciones complejas.	Rendimiento
RNF-2	Todas las contraseñas deben almacenarse con hash bcrypt. Ninguna contraseña en texto plano puede persistir en la base de datos.	Seguridad
RNF-3	Los tokens JWT deben firmarse con un secreto de al menos 64 bytes en Base64.	Seguridad
RNF-4	La API debe documentarse automáticamente y ser ejecutable desde Swagger UI en <code>/swagger-ui.html</code> .	Mantenibilidad
RNF-5	El sistema de almacenamiento de imágenes debe estar abstraído detrás de una interfaz que permita migrar de almacenamiento local a S3 sin cambiar la API pública.	Escalabilidad
RNF-6	El frontend debe funcionar en dispositivos Android (API 21+) e iOS (13+) sin bifurcaciones de código específicas por plataforma en la lógica de negocio.	Portabilidad
RNF-7	La aplicación debe operar en modo oscuro por defecto.	Usabilidad
RNF-8	Las migraciones de base de datos deben versionarse mediante Flyway y ejecutarse automáticamente al arrancar.	Fiabilidad
RNF-9	El sistema de caché HTTP de imágenes debe configurar cabeceras de expiración de un año para reducir tráfico.	Rendimiento
RNF-10	El CORS debe ser configurable por variable de entorno para no exponer orígenes abiertos (*) en producción.	Seguridad
RNF-11	El sistema debe soportar al menos 1000 usuarios concurrentes en condiciones normales de uso sin degradación crítica del servicio.	Rendimiento

Continúa en la siguiente página

Continuación de la tabla anterior

ID	Descripción	Categoría
RNF-12	Los listados de negocios, servicios, reservas y valoraciones deben implementar paginación para evitar la carga masiva de registros en una única petición.	Rendimiento
RNF-13	Las operaciones de búsqueda y filtrado deben estar optimizadas mediante consultas eficientes e índices adecuados en base de datos.	Rendimiento
RNF-14	Las tareas no críticas, como el envío de notificaciones push o procesos programados, deben ejecutarse sin bloquear la respuesta principal de la API.	Rendimiento
RNF-15	La aplicación debe mostrar estados de carga, mensajes de error y estados vacíos cuando no existan datos disponibles.	Usabilidad
RNF-16	Los formularios de registro, inicio de sesión, reserva y valoración deben validar los datos introducidos y mostrar mensajes comprensibles para el usuario.	Usabilidad
RNF-17	Los flujos principales de la aplicación, como iniciar sesión, buscar negocios, reservar un servicio y publicar una valoración, deben poder completarse con el menor número posible de pasos.	Usabilidad
RNF-18	La interfaz debe adaptarse correctamente a diferentes tamaños de pantalla en dispositivos móviles.	Usabilidad
RNF-19	El servicio debe estar disponible al menos el 99 % del tiempo mensual, excluyendo mantenimientos planificados.	Disponibilidad
RNF-20	La API debe gestionar los errores internos de forma controlada, devolviendo respuestas JSON homogéneas y evitando exponer trazas técnicas al cliente.	Fiabilidad
RNF-21	La aplicación debe poder seguir funcionando parcialmente aunque servicios externos, como Firebase Cloud Messaging, no estén disponibles temporalmente.	Disponibilidad
RNF-22	El sistema debe registrar los errores relevantes para facilitar su diagnóstico y resolución durante el mantenimiento.	Observabilidad
RNF-23	Los endpoints protegidos deben validar el rol y permisos del usuario antes de permitir operaciones sensibles.	Seguridad

Continúa en la siguiente página

Continuación de la tabla anterior

ID	Descripción	Categoría
RNF-24	Los endpoints de autenticación deben aplicar limitación de peticiones para reducir ataques de fuerza bruta.	Seguridad
RNF-25	La comunicación entre cliente y servidor debe realizarse mediante HTTPS en entornos de producción.	Seguridad
RNF-26	Las claves, secretos, credenciales y configuraciones sensibles deben almacenarse mediante variables de entorno y no directamente en el código fuente.	Seguridad
RNF-27	Los refresh tokens deben almacenarse de forma segura y permitir la renovación de sesión sin requerir que el usuario vuelva a iniciar sesión continuamente.	Seguridad
RNF-28	Los datos recibidos por la API deben validarse antes de ser procesados o almacenados en la base de datos.	Seguridad
RNF-29	El sistema debe cumplir con el RGPD y la LOPD GDD en el tratamiento de datos personales de usuarios, negocios y trabajadores.	Privacidad
RNF-30	Solo deben solicitarse y almacenarse los datos personales estrictamente necesarios para el funcionamiento de la aplicación.	Privacidad
RNF-31	Los datos personales de los usuarios no deben mostrarse públicamente salvo que sean necesarios para una funcionalidad concreta.	Privacidad
RNF-32	Los tokens y datos de sesión del usuario deben almacenarse de forma segura en el dispositivo móvil.	Privacidad
RNF-33	La API debe mantener una estructura de respuestas JSON consistente para facilitar su consumo desde el frontend y futuras integraciones.	Compatibilidad
RNF-34	La URL base de la API debe poder configurarse según el entorno de ejecución: desarrollo local, emulador, red local o producción.	Portabilidad
RNF-35	El backend debe organizarse en capas diferenciadas, separando controladores, servicios, repositorios, entidades, DTOs y configuración.	Mantenibilidad
RNF-36	El frontend debe mantener una arquitectura modular, separando vistas, modelos, repositorios, servicios, navegación y estado de la aplicación.	Mantenibilidad
RNF-37	El código fuente debe estar versionado en GitHub y seguir una estructura clara que facilite la colaboración y revisión de cambios.	Mantenibilidad

Continúa en la siguiente página

Continuación de la tabla anterior

ID	Descripción	Categoría
RNF-38	La documentación técnica debe permitir comprender la instalación, configuración y ejecución del backend y del frontend.	Mantenibilidad
RNF-39	La base de datos debe mantener integridad referencial mediante claves primarias, claves foráneas y restricciones cuando sea necesario.	Fiabilidad
RNF-40	El sistema no debe permitir que un usuario valore una reserva que no le pertenece o que no haya finalizado correctamente.	Fiabilidad
RNF-41	El sistema de reservas debe respetar la disponibilidad real de servicios, trabajadores, horarios y bloqueos existentes.	Fiabilidad
RNF-42	Las operaciones críticas, como creación de reservas, cancelaciones y publicación de valoraciones, deben evitar estados inconsistentes en la base de datos.	Fiabilidad
RNF-43	El sistema debe poder crecer en número de usuarios, negocios, servicios, reservas, valoraciones e imágenes sin requerir una reestructuración completa.	Escalabilidad
RNF-44	La arquitectura debe permitir incorporar nuevas funcionalidades, como CRM, facturación, estadísticas o panel de empresa, sin afectar gravemente a los módulos existentes.	Escalabilidad
RNF-45	La API debe exponer endpoints REST claros y documentados para permitir su consumo desde la app móvil y posibles clientes futuros.	Interoperabilidad
RNF-46	El sistema debe permitir la integración con servicios externos, como Firebase Cloud Messaging, para el envío de notificaciones push.	Interoperabilidad
RNF-47	Los textos visibles para el usuario deben estructurarse de forma que puedan traducirse en futuras versiones sin reescribir la interfaz.	Internacionalización
RNF-48	La interfaz debe utilizar tamaños de texto, contrastes y elementos visuales adecuados para facilitar su uso en dispositivos móviles.	Accesibilidad
RNF-49	Los botones, formularios y elementos interactivos deben ser fácilmente identificables y utilizables mediante interacción táctil.	Accesibilidad
RNF-50	Los mensajes importantes no deben depender exclusivamente del color, sino que deben acompañarse de texto o iconografía clara.	Accesibilidad
RNF-51	Los datos como correos electrónicos, teléfonos, fechas, precios y duraciones deben validarse antes de ser persistidos.	Calidad de datos

Continúa en la siguiente página

Continuación de la tabla anterior

ID	Descripción	Categoría
RNF-52	El sistema debe evitar duplicidades indebidas en entidades críticas, como usuarios registrados, reservas o valoraciones asociadas a una misma reserva.	Calidad de datos
RNF-53	Las fechas y horas de reservas deben gestionarse de forma coherente para evitar conflictos de disponibilidad.	Calidad de datos
RNF-54	Los fallos de autenticación, conexión con base de datos, subida de imágenes o envío de notificaciones deben quedar registrados en logs técnicos.	Observabilidad
RNF-55	En futuras funcionalidades de facturación, pagos o CRM, el sistema deberá contemplar los requisitos legales específicos asociados a dichas operaciones.	Legalidad

2.4. Casos de Uso Principales

2.4.1. CU-01: Registro de Cliente

CU-01 — Registro de Cliente

Actor: Usuario no registrado.

Precondición: El usuario no tiene cuenta en el sistema.

Flujo principal:

1. El usuario abre la app y navega a la pantalla de registro.
2. Introduce nombre completo, email, nombre de usuario, teléfono y contraseña con confirmación.
3. El sistema valida que el email y el nombre de usuario no estén ya en uso.
4. El sistema crea la cuenta, genera un par de tokens y redirige al usuario al flujo principal de la app.

Flujo alternativo (datos duplicados): Si el email o el nombre de usuario ya existen, se muestra un error específico indicando el campo en conflicto.

Postcondición: El usuario dispone de cuenta activa y sesión iniciada.

2.4.2. CU-02: Exploración y Descubrimiento de Negocios

CU-02 — Exploración de Negocios

Actor: Cualquier usuario (autenticado o no).

Precondición: La API está disponible.

Flujo principal:

1. El usuario accede a la pestaña «Explorar».
2. La app carga las categorías disponibles y la primera página de negocios.

3. El usuario puede seleccionar una categoría para filtrar, realizar una búsqueda por texto libre o combinar ambos filtros.
4. La app implementa paginación infinita: al llegar al final de la lista se cargan más resultados.
5. El usuario toca una tarjeta de negocio y accede al detalle.

Postcondición: El usuario visualiza el detalle del negocio seleccionado.

2.4.3. CU-03: Consulta de Disponibilidad

CU-03 — Consulta de Disponibilidad

Actor: Cualquier usuario.

Flujo principal:

1. Desde el detalle de un negocio, el usuario selecciona un servicio y pulsa «Reservar».
2. La app solicita al backend la disponibilidad del servicio para la fecha seleccionada (`GET /services/{id}/availability?date=YYYY-MM-DD`).
3. El backend devuelve una lista de slots horarios de 15 minutos indicando disponibilidad y, en caso de indisponibilidad, la causa (`BOOKED`, `OUT_OF_SCHEDULE`, `TIME_OFF`, etc.).
4. El usuario selecciona un slot disponible y procede a confirmar la reserva (requiere autenticación).

Flujo alternativo (negocio con múltiples trabajadores): Si no se especifica trabajador, el backend agrega la disponibilidad de todos los trabajadores habilitados para ese servicio y devuelve los IDs de los disponibles en cada slot.

2.4.4. CU-04: Dejar Valoración

CU-04 — Dejar Valoración

Actor: Cliente autenticado.

Precondición: El cliente tiene al menos una reserva en estado `CONFIRMED` con fecha de fin pasada y sin valoración previa.

Flujo principal:

1. El cliente accede a «Mis citas» y selecciona una cita completada.
2. Introduce una puntuación de 1 a 5 estrellas y un comentario opcional.
3. El sistema valida que la reserva pertenece al cliente, está finalizada y no tiene valoración previa.
4. La valoración se persiste y los agregados del negocio y servicio se actualizan.

Flujo alternativo (valoración duplicada): El sistema devuelve un error `409 Conflict` indicando que la reserva ya tiene valoración.

2.4.5. CU-05: Crear Reserva

CU-05 — Crear Reserva

Actor: Cliente autenticado.

Precondición: El servicio está activo, existe un slot disponible y el cliente ha iniciado sesión.

Flujo principal:

1. El cliente selecciona un servicio, fecha, hora y, si aplica, un trabajador concreto.
2. La app envía la petición de creación al backend mediante `POST /bookings`.
3. El backend valida que la hora esté alineada a slots de 15 minutos, que la fecha no supere el horizonte de reserva del negocio y que no existan solapes con otras citas.
4. El backend comprueba además el horario configurado, las ausencias registradas y que el trabajador pertenezca al negocio y esté habilitado para ese servicio.
5. Si todas las validaciones son correctas, la reserva se crea con estado `PENDING` o `CONFIRMED` según la política de autoaceptación del negocio o trabajador.

Flujo alternativo (slot inválido o conflicto): Si la franja ya no está disponible, el trabajador no presta ese servicio o existe una ausencia, el sistema rechaza la operación indicando la causa.

Postcondición: La reserva queda registrada y asociada al cliente, negocio, servicio y trabajador cuando corresponda.

2.4.6. CU-06: Gestionar Favoritos

CU-06 — Añadir o Quitar Favoritos

Actor: Cliente autenticado.

Precondición: El cliente está autenticado y visualiza el detalle de un negocio o servicio existente.

Flujo principal:

1. El cliente pulsa el icono de favorito desde la ficha de un negocio o de un servicio.
2. La app envía la operación al backend mediante `PUT /customers/me/favorites/businesses/{id}` o `PUT /customers/me/favorites/services/{id}`.
3. El backend añade la referencia al conjunto de favoritos del cliente y devuelve el perfil actualizado.
4. La interfaz refleja el nuevo estado visual y el elemento pasa a estar disponible en las listas personalizadas de favoritos.

Flujo alternativo (eliminación): Si el elemento ya estaba marcado, la app puede revertir la acción mediante `DELETE` sobre el mismo recurso.

Postcondición: El cliente mantiene una colección persistente de negocios y servicios favoritos.

2.4.7. CU-07: Cancelar Reserva

CU-07 — Cancelación de Reserva

Actor: Cliente autenticado, trabajador asignado o negocio propietario.

Precondición: La reserva existe y el actor actual tiene permisos para verla y modificarla.

Flujo principal:

1. El usuario accede al detalle de una reserva activa y selecciona la opción de cancelar.
2. Introduce opcionalmente un motivo y la app llama a `PUT /bookings/{id}/cancel`.
3. El backend verifica que el actor actual puede cancelar esa reserva y actualiza su estado a `CANCELED`.
4. El sistema registra el motivo, el actor que ejecutó la cancelación y publica el evento correspondiente para notificaciones.

Flujo alternativo (cancelación sin motivo): Si no se informa una razón, la reserva se cancela igualmente y el campo `cancelReason` queda vacío.

Postcondición: La reserva permanece en el histórico con trazabilidad de quién la canceló y con estado final `CANCELED`.

2.4.8. CU-08: Vincular Trabajador a un Negocio

CU-08 — Invitación y Vinculación de Trabajador

Actor: Empresa autenticada y trabajador autenticado.

Precondición: La empresa tiene sesión activa y el trabajador no está vinculado ya a otro negocio.

Flujo principal:

1. La empresa accede a la gestión de equipo y crea una invitación mediante `POST /invitations`.
2. El backend genera un código único de seis dígitos con caducidad y lo asocia al negocio emisor.
3. El trabajador introduce o consulta el código y la app valida la invitación con `GET /invitations/token/{token}`.
4. Si la invitación es válida, el trabajador la canjea con `POST /invitations/redeem/{token}`.
5. El backend vincula al trabajador con el negocio y marca la invitación como utilizada.

Flujo alternativo (token expirado o ya usado): El sistema rechaza el canje e informa de que la invitación ya no es válida.

Postcondición: El trabajador pasa a formar parte del equipo del negocio.

2.4.9. CU-09: Gestionar Ausencias del Equipo

CU-09 — Solicitud y Gestión de Ausencias

Actor: Trabajador autenticado y empresa autenticada.

Precondición: El trabajador está vinculado a un negocio y existe un calendario operativo configurado.

Flujo principal:

1. El trabajador registra una solicitud de ausencia indicando tipo, rango de fechas y motivo mediante `POST /absences`.
2. La empresa consulta las ausencias pendientes desde `GET /absences/business` y revisa sus posibles conflictos.
3. La interfaz puede recuperar las citas afectadas con `GET /absences/{id}/conflicts` antes de tomar una decisión.
4. La empresa aprueba o rechaza la solicitud usando los endpoints `/approve` o `/reject`.

Flujo alternativo (creación directa por la empresa): El negocio puede registrar una ausencia directamente para un trabajador mediante `POST /absences/direct`.

Postcondición: La ausencia queda registrada con estado `PENDING`, `APPROVED` o `REJECTED`, según corresponda.

2.5. Modelo de Dominio Conceptual

Para mejorar la legibilidad, el modelo se presenta en dos vistas complementarias derivadas de las entidades JPA reales del backend: una vista núcleo centrada en actores, catálogo, reservas y valoraciones, y una vista de soporte operativo con horarios, ausencias, invitaciones y tokens.

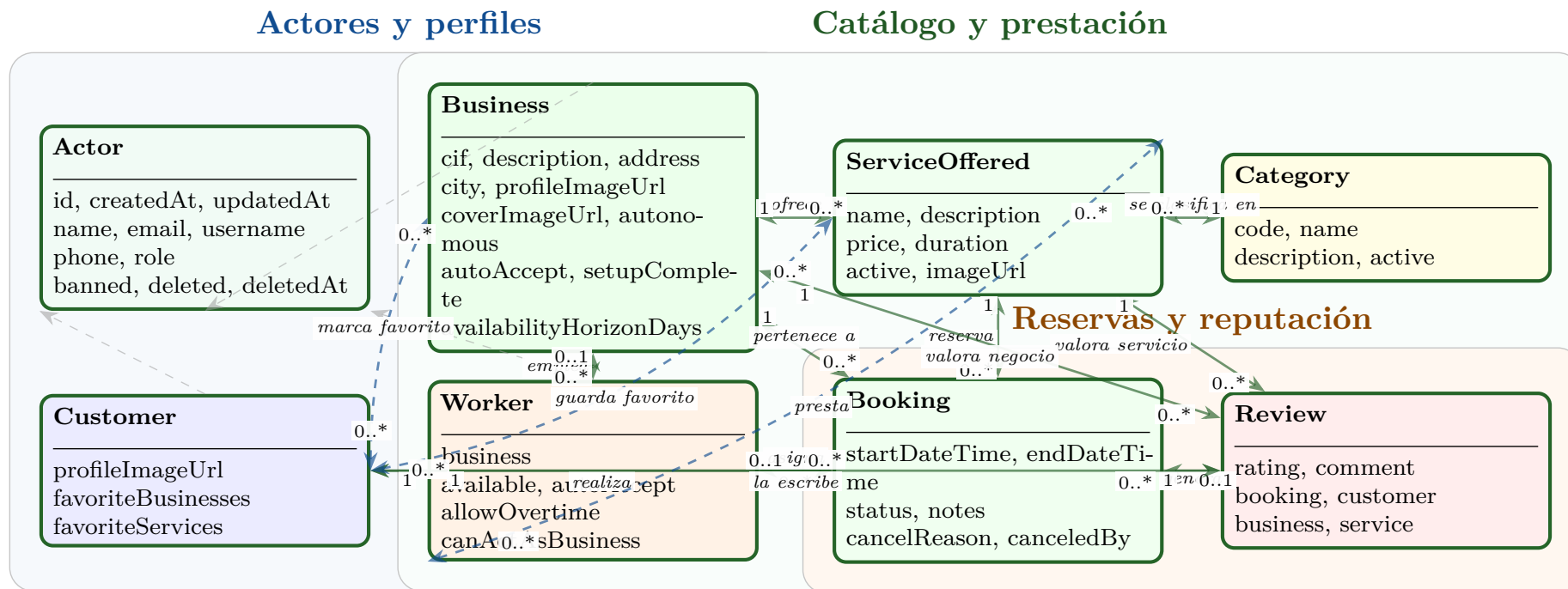


Figura 2.1: Vista núcleo del modelo de dominio de Gipsi

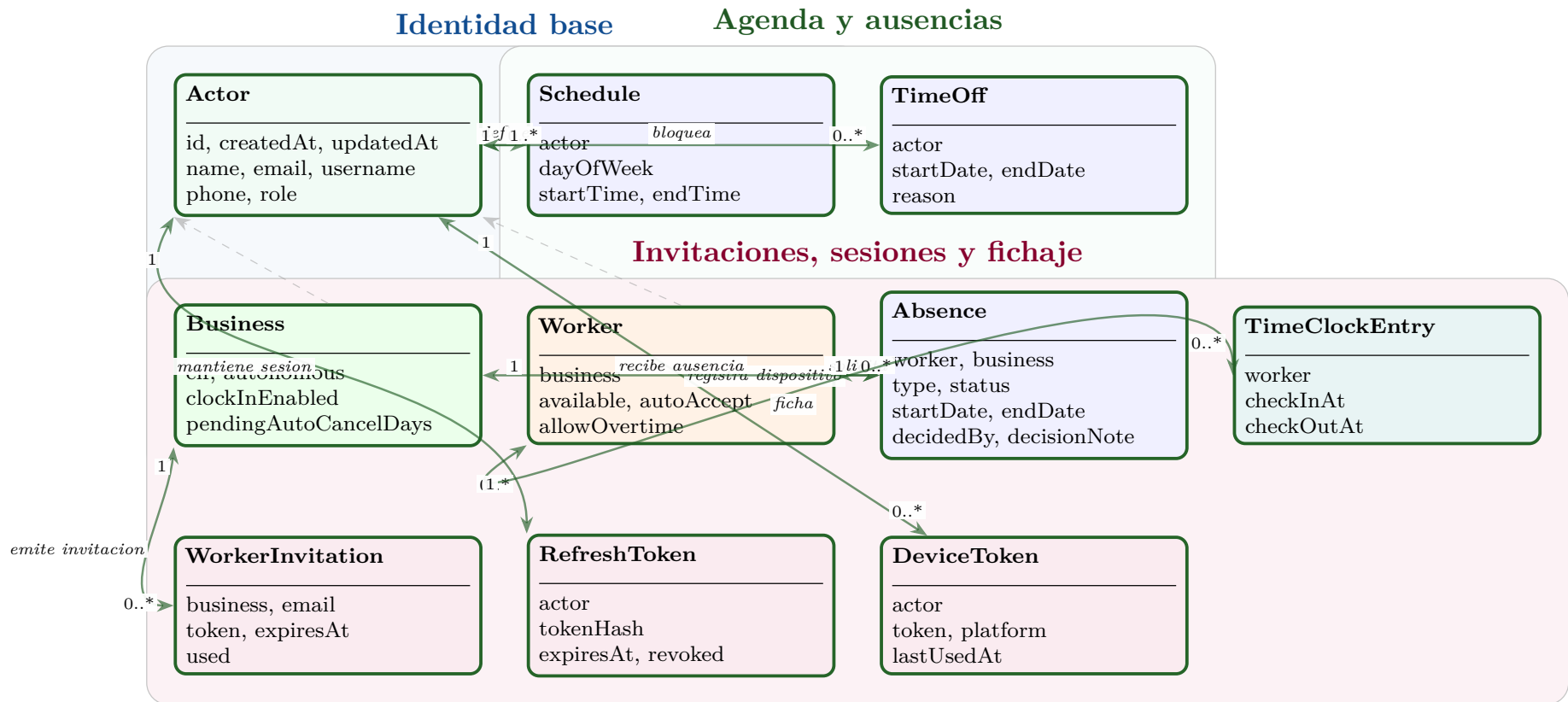


Figura 2.2: Vista operativa y de soporte del modelo de dominio

Las entidades **Customer**, **Business** y **Worker** extienden **Actor**, que concentra las credenciales y metadatos comunes. En la implementación actual la herencia JPA se resuelve mediante **InheritanceType.JOINED**, lo que separa los atributos específicos de cada subtipo en tablas propias sin perder una identidad compartida en la tabla base.

Además, los favoritos del cliente se materializan mediante relaciones muchos a muchos hacia **Business** y **ServiceOffered**, mientras que el catálogo de servicios y la agenda operativa quedan conectados con el motor de reservas a través de **ServiceOffered**, **Schedule**, **TimeOff** y **Absence**.

Capítulo 3

Diseño

3.1. Arquitectura General del Sistema

Gipsi sigue una arquitectura cliente-servidor clásica con separación física entre la capa de presentación (Flutter) y la capa de lógica de negocio y datos (Spring Boot + PostgreSQL). La comunicación entre ambas se realiza exclusivamente mediante HTTP/HTTPS siguiendo el estilo arquitectónico REST.

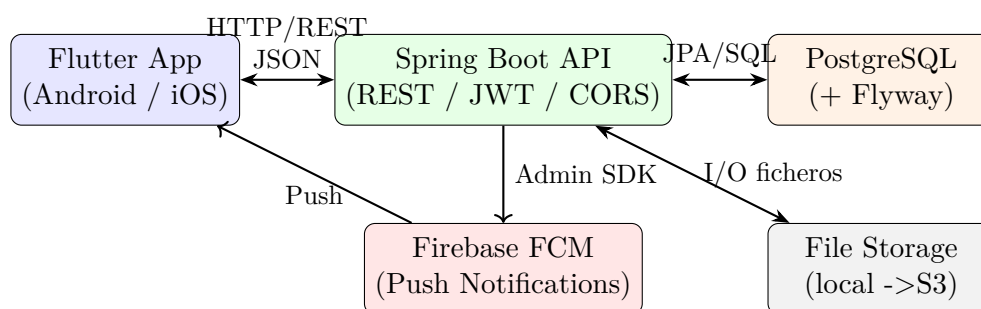


Figura 3.1: Arquitectura general del sistema Gipsi

3.1.1. Principios Arquitectónicos Aplicados

Separación de responsabilidades. El backend organiza su código en capas: controladores REST (*Controller*), lógica de negocio (*Service*), acceso a datos (*Repository*) y objetos de transferencia (*DTO*). Ninguna lógica de negocio reside en los controladores; ninguna consulta SQL manual aparece fuera de los repositorios.

Principio de responsabilidad única. Cada clase tiene una única razón de cambio. Por ejemplo, `JwtService` solo se ocupa de generación y validación de tokens; `LocalFileStorage` solo gestiona operaciones de disco; `AvailabilityService` solo calcula disponibilidad horaria.

Inversión de dependencias. La interfaz `FileStorage` actúa como contrato entre el servicio de negocio y la implementación concreta de almacenamiento. Hoy se usa `LocalFileStorage`; en el futuro puede sustituirse por `S3FileStorage` sin modificar ningún servicio.

Stateless API. El servidor no mantiene estado de sesión entre peticiones. Toda la información de autenticación viaja en la cabecera `Authorization: Bearer <token>`, y el estado de la sesión se reconstituye en cada petición a partir del JWT.

3.2. Arquitectura del Backend (Spring Boot)

3.2.1. Estructura de Paquetes

El proyecto Maven sigue la convención estándar de Spring Boot con el groupId `com.gipsi`:

```
src/main/java/com/gipsi/gipsibackend/
|-- GipsiBackendApplication.java      # Punto de entrada
|-- config/
|   |-- SecurityConfig.java          # Filtros, CORS, reglas de acceso
|   |-- CorsConfig.java              # Configuración CORS por entorno
|   |-- FirebaseConfig.java          # Inicialización Firebase Admin SDK
|-- auth/
|   |-- controller/AuthController.java
|   |-- service/AuthService.java
|   |-- service/JwtService.java
|   |-- filter/JwtAuthFilter.java
|   |-- dto/ (LoginRequest, RegisterRequest, TokenResponse...)
|-- user/
|   |-- model/ (User, Customer, Business, Worker)
|   |-- repository/UserRepository.java
|-- business/
|   |-- controller/BusinessController.java
|   |-- service/BusinessService.java
|   |-- model/Business.java
|   |-- repository/BusinessRepository.java
|   |-- dto/ (BusinessRequest, BusinessResponse...)
|-- service/                          # Paquete de servicios ofrecidos
|   |-- controller/ServiceController.java
|   |-- service/ServiceOfferedService.java
|   |-- service/AvailabilityService.java
|   |-- model/ServiceOffered.java
|   |-- dto/
|-- booking/
|   |-- controller/BookingController.java
|   |-- service/BookingService.java
|   |-- model/ (Booking, BookingStatus)
|   |-- dto/
|-- review/
|   |-- controller/ReviewController.java
|   |-- service/ReviewService.java
|   |-- model/Review.java
|   |-- dto/
|-- notification/
|   |-- service/NotificationService.java
|-- storage/
|   |-- FileStorage.java              # Interfaz
|   |-- LocalFileStorage.java
|   |-- S3FileStorage.java           # Esqueleto futuro
|-- category/
|   |-- model/Category.java
```

```
| |-- repository/CategoryRepository.java
|-- ratelimit/
| |-- RateLimitFilter.java           # Bucket4j
|-- common/
| |-- DataInitializer.java           # Datos de prueba (perfil dev)
| |-- GlobalExceptionHandler.java
|-- scheduler/
| |-- BookingScheduler.java          # Jobs: auto-cancel + recordatorios
```

Listing 3.1: Estructura de paquetes del backend

3.2.2. Capa de Seguridad: JWT y Spring Security

La seguridad es el aspecto más crítico del backend. Se implementa mediante una cadena de filtros de Spring Security que intercepta cada petición HTTP antes de que llegue a cualquier controlador.

¿Qué es un JSON Web Token?

Un JSON Web Token (JWT) es un estándar abierto (RFC 7519) para transmitir información entre partes de forma compacta y autocontenida. Se compone de tres partes codificadas en Base64URL separadas por puntos:

1. **Header:** tipo del token y algoritmo de firma (HS256 en Gipsi).
2. **Payload:** las *claims* o afirmaciones (subject = username, roles, fecha de expiración).
3. **Signature:** firma HMAC-SHA256 del header y payload usando el secreto del servidor.

Puesto que el servidor firma el token, cualquier modificación del payload hace que la firma sea inválida. Esto permite verificar la autenticidad del token **sin consultar la base de datos** en cada petición, lo que aporta escalabilidad. La contrapartida es que los tokens no pueden invalidarse antes de su expiración (de ahí la necesidad del refresh token con revocación en base de datos).

En Gipsi no se utiliza un único token de sesión, sino un par formado por *access token* y *refresh token*. Esta separación responde a una necesidad de equilibrio entre seguridad y experiencia de usuario: el sistema debe poder autenticar cada petición de forma rápida, pero también debe evitar que una credencial robada permita acceder indefinidamente a la cuenta.

Access token. El *access token* es el token que se envía en las peticiones normales a la API, dentro de la cabecera **Authorization: Bearer <token>**. Su función es demostrar que el usuario ya se ha autenticado y que puede acceder a un recurso concreto. En Gipsi tiene una duración corta, de 30 minutos, porque viaja con frecuencia entre la app y el backend. Si alguien llegara a interceptarlo, el impacto quedaría limitado a una ventana temporal reducida. Además, al ser un JWT firmado, el servidor puede validarlo de manera local comprobando su firma, su fecha de expiración y sus *claims*, sin consultar la base de datos en cada petición.

Refresh token. El *refresh token*, en cambio, no se utiliza para llamar a endpoints funcionales como reservas, negocios o perfiles. Su único propósito es obtener un nuevo

par de tokens cuando el *access token* caduca. Por eso dura más tiempo, 30 días, y se trata como una credencial más sensible. A diferencia del *access token*, el *refresh token* sí se guarda en la base de datos, asociado al usuario, con información de expiración y revocación. Esto permite invalidarlo cuando el usuario cierra sesión, cuando se detecta un uso sospechoso o cuando se rota por uno nuevo.

La existencia de dos tokens evita dos problemas habituales. Si solo existiera un token corto, el usuario tendría que iniciar sesión constantemente, lo que haría la aplicación incómoda. Si solo existiera un token largo usado en todas las peticiones, cualquier robo de esa credencial tendría consecuencias graves, porque permitiría acceder a la API durante mucho tiempo. Con el par *access/refresh*, el acceso diario es ágil y stateless, mientras que la continuidad de la sesión queda controlada mediante un token revocable y almacenado de forma segura.

El comportamiento práctico es el siguiente: tras un login correcto, el backend devuelve ambos tokens. La app guarda el par en almacenamiento seguro y usa el *access token* en cada petición autenticada. Cuando una petición falla porque el *access token* ha caducado, el interceptor HTTP llama a `POST /auth/refresh` con el *refresh token*. Si este sigue siendo válido, el backend emite un nuevo *access token* y un nuevo *refresh token*; la app los guarda y reintenta la petición original sin obligar al usuario a volver a escribir sus credenciales. Si el *refresh token* ha expirado, ha sido revocado o ya fue utilizado anteriormente, la sesión se considera inválida y el usuario debe autenticarse de nuevo.

El flujo de autenticación completo se representa en el diagrama de la Figura 3.2.

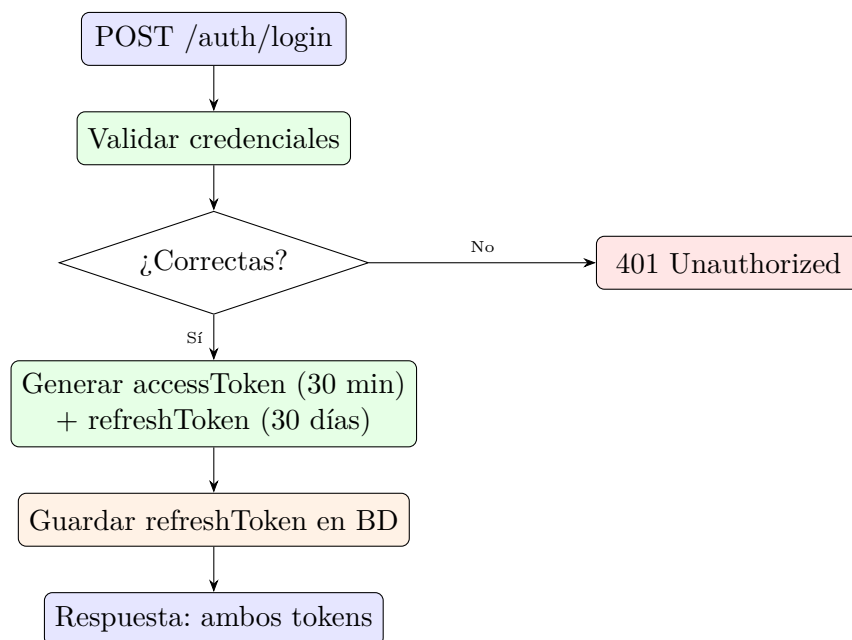


Figura 3.2: Flujo de autenticación en el backend

Rotación de Refresh Tokens. La característica más relevante del sistema de autenticación es la rotación de refresh tokens, implementada siguiendo las recomendaciones de OWASP. Cuando el cliente presenta un refresh token para obtener un nuevo par, el sistema:

1. Verifica que el token existe en la base de datos y no ha sido revocado.
2. Emite un nuevo *access token* y un nuevo *refresh token*.
3. **Invalida el refresh token anterior** en la base de datos.
4. Si alguien intenta usar el token antiguo (lo que indicaría que fue robado), el sistema detecta que ya fue rotado y **revoca toda la cadena de tokens del usuario**, forzando un nuevo login.

Este mecanismo ofrece protección ante el robo de refresh tokens sin necesidad de mantener una lista negra de access tokens.

3.2.3. Cálculo de Disponibilidad Horaria

El cálculo de disponibilidad es la funcionalidad más compleja del backend. El `AvailabilityService` genera slots de 15 minutos y para cada slot evalúa las siguientes condiciones:

1. **OUT_OF_SCHEDULE**: el slot cae fuera del horario de apertura del negocio para ese día de la semana.
2. **TIME_OFF**: el trabajador tiene registrado un día libre o ausencia que cubre ese slot.
3. **OVERTIME_NOT_ALLOWED**: el servicio terminaría después del cierre del negocio y la política no permite horas extra.
4. **BOOKED**: existe una reserva confirmada o pendiente del mismo trabajador que se solapa con el slot.
5. **WORKER_UNAVAILABLE**: ningún trabajador habilitado para ese servicio está disponible (en el modo multi-worker).

```

1 // Por cada slot del día (granularidad: 15 min)
2 for (LocalTime start = openTime;
   start.plusMinutes(serviceDuration).compareTo(closeTime) <= 0;
   start = start.plusMinutes(SLOT_GRANULARITY)) {
3
4
5     LocalTime end = start.plusMinutes(serviceDuration);
6     SlotReason reason = SlotReason.AVAILABLE;
7
8     if (!isWithinSchedule(worker, date, start, end)) {
9         reason = SlotReason.OUT_OF_SCHEDULE;
10    } else if (hasTimeOff(worker, date)) {
11        reason = SlotReason.TIME_OFF;
12    } else if (hasConflictingBooking(worker, date, start, end)) {
13        reason = SlotReason.BOOKED;
14    }
15
16    slots.add(new SlotDto(start, end, reason == SlotReason.AVAILABLE, reason));
17 }

```

Listing 3.2: Fragmento conceptual del cálculo de disponibilidad

3.3. Arquitectura del Frontend (Flutter)

3.3.1. Monorepo con Melos

El frontend se organiza como monorepo gestionado por **Melos**, una herramienta diseñada para proyectos Dart/Flutter con múltiples packages interdependientes. La estructura es:

```
gipsi-frontend/
|-- apps/
|   |-- gipsi/                                # App ejecutable (Android + iOS)
|       |-- lib/
|           |-- main.dart
|           |-- router/                        # Configuración go_router
|           |-- features/
|               |-- explore/                  # Pantalla de exploración
|               |-- business/                 # Detalle de negocio
|               |-- auth/                    # Login, registro, perfil
|               |-- bookings/                # Mis citas (pendiente)
|               |-- shell/                   # Bottom nav + ShellRoute
|           |-- providers/                   # Riverpod providers globales
|       |-- assets/
|           |-- logo/
|-- packages/
|   |-- gipsi_api/                            # Cliente HTTP, modelos, repositorios
|       |-- lib/src/
|           |-- api_client.dart              # Dio + interceptor refresh
|           |-- models/                     # Clases Dart de dominio
|           |-- repositories/                # Contratos y implementaciones
|   |-- gipsi_shared_ui/                     # Sistema de diseño
|       |-- lib/src/
|           |-- theme/                      # GipsiTheme (claro + oscuro)
|           |-- tokens/                    # Colores, tipografía, espaciado
|           |-- components/                 # Widgets reutilizables
|           |-- logo/                      # GipsiLogo widget
|-- melos.yaml
|-- pubspec.yaml                             # Raíz del workspace
```

Listing 3.3: Estructura del monorepo Flutter

Ventajas del monorepo con Melos. Separar el código en packages (en lugar de directorios dentro de una sola app) aporta varias ventajas concretas:

- `gipsi_api` puede testearse de forma completamente independiente de la UI, incluso en entornos sin Flutter (Dart puro).
- `gipsi_shared_ui` puede compartirse con futuras apps del ecosistema Gipsi (app de negocio, app de trabajador) sin duplicar código.
- Los cambios en un package quedan aislados: modificar el cliente HTTP no fuerza recompilación de los tests de UI.

3.3.2. Gestión de Estado con Riverpod

Riverpod es el sistema de gestión de estado elegido para Gipsi. Es la evolución madura del paquete Provider, diseñada para eliminar sus limitaciones principales (dependencia

del árbol de widgets, ausencia de capacidades de computación asíncrona, dificultad para combinar providers).

¿Por qué Riverpod y no BLoC o setState?

setState Es adecuado para estado local y efímero (animaciones, expansión de un panel). No escala cuando el estado debe compartirse entre pantallas distantes en el árbol de widgets.

BLoC Potente y muy estructurado, pero introduce considerable boilerplate (eventos, estados, mappers) que puede ser excesivo para proyectos de tamaño medio.

Riverpod Ofrece un equilibrio: providers que pueden ser asíncronos (`FutureProvider`, `StreamProvider`), reactivos (`NotifierProvider`) y composables (un provider puede depender de otro). El código de Gipsi usa `AsyncNotifier` para los controladores de auth y exploración, lo que simplifica enormemente el manejo de estados cargando/error/datos.

3.3.3. Navegación con go_router

La navegación se implementa con `go_router`, que define las rutas de forma declarativa y permite la integración con el estado de autenticación mediante *redirects* reactivos. La estructura de rutas utiliza un `ShellRoute` para el bottom navigation bar, de modo que las cuatro pestañas principales (Explorar, Buscar, Mis citas, Perfil) comparten el mismo scaffold mientras sus contenidos se sustituyen.

```

1 final router = GoRouter(
2   redirect: (context, state) {
3     final isAuthenticated = ref.read(authProvider).isAuthenticated;
4     final requiresAuth = ['/bookings'].contains(state.matchedLocation);
5     if (requiresAuth && !isAuthenticated) return '/login';
6     return null;
7   },
8   routes: [
9     ShellRoute(
10      builder: (_, __, child) => AppShell(child: child),
11      routes: [
12        GoRoute(path: '/explore', builder: (_, __) => ExplorePage()),
13        GoRoute(path: '/search', builder: (_, __) => SearchPage()),
14        GoRoute(path: '/bookings', builder: (_, __) => BookingsPage()),
15        GoRoute(path: '/profile', builder: (_, __) => ProfilePage()),
16      ],
17    ),
18    GoRoute(path: '/business/:id', builder: (_, s) =>
19      BusinessDetailPage(id: s.pathParameters['id']!)),
20    GoRoute(path: '/login', builder: (_, __) => LoginPage()),
21    GoRoute(path: '/register', builder: (_, __) => RegisterPage()),
22  ],
23 );

```

Listing 3.4: Esquema simplificado de rutas con `go_router`

3.3.4. Sistema de Diseño: gipsi_shared_ui

El package `gipsi_shared_ui` centraliza toda la identidad visual de la aplicación. Esto garantiza que cualquier cambio en el tema (colores, tipografía, espaciado) se propaga automáticamente a todos los widgets sin modificaciones individuales.

Tabla 3.1: Tokens de color del sistema de diseño Gipsi

Token	Hex	Uso	Modo
<code>gipsiFondo</code>	<code>#1d292f</code>	Fondo principal	Oscuro
<code>gipsiPrimario</code>	<code>#c0eed3</code>	Acento de marca (verde menta)	Ambos
<code>gipsiSecundario</code>	<code>#83a38e</code>	Elementos secundarios	Ambos
<code>gipsiContraste</code>	<code>#44544b</code>	Bordes, separadores	Ambos
<code>white</code>	<code>#ffffff</code>	Texto principal sobre oscuro	Oscuro

La tipografía utilizada en toda la aplicación es **Montserrat**, servida mediante el paquete `google_fonts`, que la descarga y cachea automáticamente en el primer uso.

3.4. Modelo de Datos Relacional

3.4.1. Esquema Principal

```

1  -- Tabla base de usuarios (esquema simplificado de una versión temprana)
2  CREATE TABLE users (
3      id          BIGSERIAL PRIMARY KEY,
4      dtype       VARCHAR(20) NOT NULL, -- 'CUSTOMER', 'BUSINESS', 'WORKER'
5      username    VARCHAR(50)  UNIQUE NOT NULL,
6      email       VARCHAR(100) UNIQUE NOT NULL,
7      password    VARCHAR(255) NOT NULL, -- bcrypt hash
8      full_name   VARCHAR(100),
9      phone       VARCHAR(20),
10     created_at   TIMESTAMP DEFAULT NOW()
11 );
12
13 -- Campos específicos de BUSINESS (nulos para otros roles)
14 ALTER TABLE users ADD COLUMN description TEXT;
15 ALTER TABLE users ADD COLUMN city        VARCHAR(100);
16 ALTER TABLE users ADD COLUMN address     VARCHAR(255);
17 ALTER TABLE users ADD COLUMN is_solo     BOOLEAN DEFAULT FALSE;
18 ALTER TABLE users ADD COLUMN profile_img VARCHAR(255);
19 ALTER TABLE users ADD COLUMN cover_img   VARCHAR(255);
20 ALTER TABLE users ADD COLUMN setup_complete BOOLEAN DEFAULT FALSE;
21
22 -- Categorías
23 CREATE TABLE categories (
24     id          BIGSERIAL PRIMARY KEY,
25     name        VARCHAR(100) UNIQUE NOT NULL,
26     active      BOOLEAN DEFAULT TRUE
27 );
28
29 -- Servicios ofrecidos
30 CREATE TABLE services_offered (
31     id          BIGSERIAL PRIMARY KEY,

```



```

32     business_id BIGINT REFERENCES users(id),
33     category_id BIGINT REFERENCES categories(id),
34     name        VARCHAR(100) NOT NULL,
35     duration_min INTEGER NOT NULL,
36     price       NUMERIC(8,2),
37     image_url   VARCHAR(255),
38     active      BOOLEAN DEFAULT TRUE
39 );
40
41 -- Reservas
42 CREATE TABLE bookings (
43     id            BIGSERIAL PRIMARY KEY,
44     customer_id   BIGINT REFERENCES users(id),
45     service_id    BIGINT REFERENCES services_offered(id),
46     worker_id     BIGINT REFERENCES users(id),
47     start_datetime TIMESTAMP NOT NULL,
48     end_datetime  TIMESTAMP NOT NULL,
49     status        VARCHAR(20) DEFAULT 'PENDING',
50     created_at    TIMESTAMP DEFAULT NOW()
51 );
52
53 -- Valoraciones
54 CREATE TABLE reviews (
55     id            BIGSERIAL PRIMARY KEY,
56     booking_id    BIGINT UNIQUE REFERENCES bookings(id),
57     customer_id   BIGINT REFERENCES users(id),
58     business_id   BIGINT REFERENCES users(id),
59     service_id    BIGINT REFERENCES services_offered(id),
60     rating        SMALLINT CHECK (rating BETWEEN 1 AND 5),
61     comment       TEXT,
62     created_at    TIMESTAMP DEFAULT NOW()
63 );
64
65 -- Refresh tokens con rotación
66 CREATE TABLE refresh_tokens (
67     id            BIGSERIAL PRIMARY KEY,
68     user_id       BIGINT REFERENCES users(id),
69     token         VARCHAR(512) UNIQUE NOT NULL,
70     revoked       BOOLEAN DEFAULT FALSE,
71     expires_at    TIMESTAMP NOT NULL,
72     created_at    TIMESTAMP DEFAULT NOW()
73 );

```

Listing 3.5: DDL simplificado de las tablas principales (Flyway V1)

3.4.2. Gestión de Migraciones con Flyway

Flyway aplica las migraciones en orden al arrancar la aplicación, lo que garantiza que el esquema de base de datos es siempre consistente con el código. El convenio de nombrado es `V{número}_{descripción}.sql`. La versión 2 del proyecto (v2) aplica `V2_improvements.sql` sobre bases de datos existentes sin necesidad de recrearlas.

Nota

Las migraciones de Flyway son **inmutables** una vez aplicadas: no se deben modificar los archivos `V1_...sql` ni `V2_...sql` una vez que están en producción. Las

correcciones se realizan siempre mediante nuevas versiones (v3__...sql).

3.5. Diseño de la API REST

3.5.1. Convenciones Generales

La API sigue las convenciones REST estándar: recursos en plural y en minúsculas (/businesses, /services), verbos HTTP semánticos (GET para consultas, POST para creación, PUT para actualización completa, PATCH para parcial, DELETE para eliminación), y códigos de respuesta HTTP estándar (200, 201, 400, 401, 403, 404, 409, 429).

3.5.2. Catálogo de Endpoints

Tabla 3.2: Endpoints de la API — Autenticación y Perfil

Método	Ruta	Descripción	Rol requerido
POST	/auth/login	Autenticación, devuelve par de tokens	Público
POST	/auth/register/customer	Registro de cliente	Público
POST	/auth/refresh	Rota refresh token	Público
POST	/auth/logout	Revoca refresh token	Autenticado
GET	/me	Perfil del usuario actual (polimórfico)	Autenticado
GET	/me/devices	Lista de dispositivos FCM registrados	Autenticado
POST	/me/devices	Registra token FCM	Autenticado

Tabla 3.3: Endpoints de la API — Negocios y Servicios

Método	Ruta	Descripción	Rol requerido
GET	/businesses	Lista paginada con filtros	Público
GET	/businesses/{id}	Detalle de negocio	Público
POST	/businesses/me/images/profile	Sube imagen de perfil	BUSINESS
POST	/businesses/me/images/cover	Sube imagen de portada	BUSINESS
GET	/categories	Lista categorías	Público
GET	/services	Lista servicios con filtros	Público
GET	/services/{id}	Detalle de servicio	Público
GET	/services/{id}/availability	Disponibilidad por fecha	Público
POST	/services/{id}/image	Sube imagen de servicio	BUSINESS

Tabla 3.4: Endpoints de la API — Reservas y Valoraciones

Método	Ruta	Descripción	Rol requerido
POST	/bookings	Crea reserva	CUSTOMER
GET	/bookings/me	Mis reservas	CUSTOMER
PUT	/bookings/{id}/cancel	Cancela reserva	CUSTOMER
POST	/reviews	Crea valoración	CUSTOMER
PUT	/reviews/{id}	Edita valoración propia	CUSTOMER
DELETE	/reviews/{id}	Elimina valoración propia	CUSTOMER
GET	/reviews/business/{bId}	Reviews de un negocio	Público
GET	/reviews/service/{sId}	Reviews de un servicio	Público
GET	/files/{cat}/{filename}	Sirve imagen estática	Público

Capítulo 4

Implementación

4.1. Backend: Detalles de Implementación

4.1.1. Configuración de Spring Security

Spring Security 6 (integrado en Spring Boot 3) adopta un modelo de configuración basado en beans en lugar de la herencia de `WebSecurityConfigurerAdapter` de versiones anteriores. La configuración de Gipsi define una cadena de filtros que:

1. Deshabilita CSRF (innecesario en APIs REST stateless, donde no hay sesiones ni cookies de sesión).
2. Configura CORS con los orígenes permitidos leídos de variables de entorno.
3. Establece la política de sesión como `STATELESS` (sin sesiones HTTP del servidor).
4. Define las reglas de autorización: endpoints públicos (exploración, imágenes, auth), endpoints por rol.
5. Registra el filtro `JwtAuthFilter` antes del filtro de autenticación estándar de Spring.

```
1 @Configuration
2 @EnableWebSecurity
3 public class SecurityConfig {
4
5     @Bean
6     public SecurityFilterChain filterChain(HttpSecurity http,
7         JwtAuthFilter jwtFilter) throws Exception {
8         return http
9             .csrf(csrf -> csrf.disable())
10            .cors(Customizer.withDefaults())
11            .sessionManagement(sm ->
12                sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
13            .authorizeHttpRequests(auth -> auth
14                .requestMatchers("/auth/**", "/businesses/**",
15                    "/services/**", "/categories/**",
16                    "/reviews/**", "/files/**",
17                    "/swagger-ui/**", "/v3/api-docs/**").permitAll()
18                .requestMatchers(HttpMethod.POST, "/bookings").hasRole("CUSTOMER")
19                .requestMatchers(HttpMethod.POST, "/reviews").hasRole("CUSTOMER")
20                .requestMatchers("/businesses/me/**").hasRole("BUSINESSS")
21            )
22        }
```

```

21         .anyRequest().authenticated()
22         .addFilterBefore(jwtFilter,
23             UsernamePasswordAuthenticationFilter.class)
24         .build();
25     }
26
27     @Bean
28     public PasswordEncoder passwordEncoder() {
29         return new BCryptPasswordEncoder();
30     }
31 }

```

Listing 4.1: Configuración de Spring Security (SecurityConfig.java)

4.1.2. Rate Limiting con Bucket4j

Para proteger los endpoints de autenticación contra ataques de fuerza bruta, Gipsi implementa rate limiting en memoria usando la librería **Bucket4j**, que implementa el algoritmo del *token bucket*.

Algoritmo Token Bucket

El token bucket es un algoritmo de control de flujo que funciona de la siguiente manera: existe un cubo que puede contener hasta N tokens. Cada petición consume un token; si el cubo está vacío, la petición se rechaza (429 Too Many Requests). El cubo se recarga a una tasa constante de r tokens por unidad de tiempo. En Gipsi, la configuración es $N = 10$ tokens con recarga de 10 tokens/minuto por dirección IP, lo que permite ráfagas cortas respetando un límite sostenido de 10 req/min.

4.1.3. Notificaciones Push con Firebase

El servicio de notificaciones (`NotificationService`) utiliza el SDK de Firebase Admin para Java para enviar mensajes FCM. Los tokens de dispositivo se registran en la entidad `DeviceToken` vinculada al usuario. El envío se realiza de forma asíncrona mediante eventos de Spring (`@EventListener + @Async`), de modo que el envío de la notificación no bloquea la respuesta HTTP principal.

```

1  @Async
2  @EventListener
3  public void onBookingCreated(BookingCreatedEvent event) {
4      Booking booking = event.getBooking();
5      List<String> tokens = deviceTokenRepository
6          .findById(booking.getBusiness().getId())
7          .stream().map(DeviceToken::getToken).toList();
8
9      if (tokens.isEmpty()) return;
10
11      MulticastMessage message = MulticastMessage.builder()
12          .setNotification(Notification.builder()
13              .setTitle("Nueva reserva")
14              .setBody("Reserva de " + booking.getCustomer().getFullName()
15                  + " para " + booking.getService().getName())
16              .build())
17          .addAllTokens(tokens)

```

```

18     .build();
19
20     try {
21         firebaseMessaging.sendEachForMulticast(message);
22     } catch (FirebaseMessagingException e) {
23         log.warn("Error FCM: {}", e.getMessage());
24     }
25 }

```

Listing 4.2: Envío asíncrono de notificación push

4.1.4. Jobs Programados

El `BookingScheduler` ejecuta dos tareas programadas:

1. **Auto-cancelación de reservas PENDING:** Se ejecuta cada hora y cancela las reservas en estado `PENDING` cuya fecha de inicio ya ha pasado sin ser confirmadas por el negocio.
2. **Recordatorios diarios:** Se ejecuta cada mañana y envía notificaciones push a los clientes con reservas confirmadas para el día siguiente.

```

1  @Scheduled(cron = "0 0 * * *") // Cada hora en punto
2  @Transactional
3  public void autoCancelExpiredPendingBookings() {
4      List<Booking> expired = bookingRepository
5          .findByStatusAndStartDateBefore(
6              BookingStatus.PENDING, LocalDateTime.now());
7      expired.forEach(b -> {
8          b.setStatus(BookingStatus.CANCELLED);
9          log.info("Auto-cancelled booking {}", b.getId());
10     });
11     bookingRepository.saveAll(expired);
12 }
13
14 @Scheduled(cron = "0 0 8 * *") // 08:00 cada día
15 public void sendDailyReminders() {
16     LocalDate tomorrow = LocalDate.now().plusDays(1);
17     List<Booking> upcoming = bookingRepository
18         .findConfirmedForDate(tomorrow);
19     upcoming.forEach(notificationService::sendReminder);
20 }

```

Listing 4.3: Job de auto-cancelación con `@Scheduled`

4.2. Frontend: Detalles de Implementación

4.2.1. Cliente HTTP con Dio e Interceptor de Refresh

El package `gipsi_api` encapsula toda la comunicación con el backend. El cliente HTTP se construye sobre **Dio**, una librería que supera a `http` en capacidades (interceptores, cancelación, progreso de subida). El interceptor de refresh token implementa la siguiente lógica:

1. Cada petición lleva la cabecera `Authorization: Bearer {accessToken}`.

2. Si la respuesta es un error 401, el interceptor captura el error antes de que llegue al código de la feature.
3. Llama a `POST /auth/refresh` con el refresh token almacenado en `flutter_secure_storage`.
4. Si el refresh tiene éxito, guarda los nuevos tokens y **reintenta la petición original** de forma transparente.
5. Si el refresh falla (token expirado o revocado), llama a `AuthController.expireSession()` que limpia el estado de auth y la UI transiciona automáticamente al estado no-autenticado.

```

1 class TokenRefreshInterceptor extends Interceptor {
2   @override
3   Future<void> onError(DioException err,
4     ErrorInterceptorHandler handler) async {
5     if (err.response?.statusCode != 401) {
6       return handler.next(err);
7     }
8     try {
9       final newTokens = await _authRepo.refresh();
10      await _storage.saveTokens(newTokens);
11      // Reintentar petición original con nuevo access token
12      final opts = err.requestOptions;
13      opts.headers['Authorization'] = 'Bearer ${newTokens.accessToken}';
14      final response = await _dio.fetch(opts);
15      return handler.resolve(response);
16    } catch (_) {
17      await _authController.expireSession();
18      return handler.reject(err);
19    }
20  }
21 }

```

Listing 4.4: Interceptor de refresh token en Dio

4.2.2. Pantalla de Exploración

La pantalla `ExplorePage` implementa varios patrones de UX avanzados:

- **Skeleton loading:** mientras carga la primera página, se muestran tarjetas con shimmer efecto para indicar carga sin un spinner bloqueante.
- **Pull-to-refresh:** el usuario puede tirar hacia abajo para forzar una recarga.
- **Paginación infinita:** al llegar al último elemento visible, se dispara la carga de la siguiente página.
- **Estado vacío:** si no hay resultados para los filtros aplicados, se muestra un mensaje específico con sugerencia de acción.
- **Estado de error:** si la petición falla, se muestra el error con un botón «Reintentar» que vuelve a lanzar la petición.

4.2.3. Pantalla de Detalle de Negocio

El detalle de negocio utiliza un `CustomScrollView` con `SliverAppBar` para conseguir el efecto de portada colapsable:

- La imagen de portada ocupa 320 px de altura y colapsa al hacer scroll, revelando el AppBar con el nombre del negocio.
- Un efecto de zoom se activa al hacer pull hacia abajo, usando el `FlexibleSpaceBar` de Flutter.
- Sobre la imagen, botones circulares semi-transparentes para volver, marcar como favorito y compartir.
- El resto del contenido (descripción, servicios, reviews) se estructura como slivers debajo.

4.2.4. Almacenamiento Seguro de Tokens

Los tokens JWT se almacenan en `flutter_secure_storage`, que utiliza el Keystore de Android (o el Keychain de iOS) para el cifrado, garantizando que los tokens no son accesibles por otras aplicaciones ni aparecen en backups no cifrados.

```
1 class SecureTokenStorage {
2   static const _storage = FlutterSecureStorage();
3   static const _accessKey = 'access_token';
4   static const _refreshKey = 'refresh_token';
5
6   Future<void> saveTokens(TokenPair tokens) async {
7     await Future.wait([
8       _storage.write(key: _accessKey, value: tokens.accessToken),
9       _storage.write(key: _refreshKey, value: tokens.refreshToken),
10    ]);
11  }
12
13  Future<TokenPair?> loadTokens() async {
14    final access = await _storage.read(key: _accessKey);
15    final refresh = await _storage.read(key: _refreshKey);
16    if (access == null || refresh == null) return null;
17    return TokenPair(accessToken: access, refreshToken: refresh);
18  }
19
20  Future<void> clear() async {
21    await Future.wait([
22      _storage.delete(key: _accessKey),
23      _storage.delete(key: _refreshKey),
24    ]);
25  }
26 }
```

Listing 4.5: Servicio de almacenamiento seguro de tokens

Capítulo 5

Pruebas y Evaluación

5.1. Estrategia de Pruebas

La estrategia de pruebas de Gipsi sigue la pirámide de testing clásica adaptada al contexto del proyecto. En la base se sitúan las pruebas unitarias (las más numerosas y rápidas), en el nivel intermedio las pruebas de integración, y en el vértice las pruebas end-to-end (manuales en esta fase del proyecto).

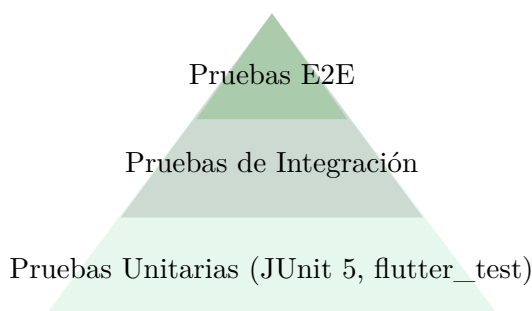


Figura 5.1: Pirámide de testing aplicada a Gipsi

Las **pruebas unitarias** verifican una pieza pequeña y aislada del sistema, normalmente una clase o un método, sustituyendo sus dependencias por *mocks* o dobles de prueba. Su objetivo es comprobar reglas de negocio concretas con la máxima rapidez posible. En este proyecto se usan para validar componentes sensibles del backend, como la generación y validación de tokens o la lógica de cálculo de disponibilidad.

Las **pruebas de integración** amplían el alcance y comprueban que varios componentes colaboran correctamente entre sí, por ejemplo controlador, servicio, repositorio, seguridad o acceso a base de datos. A diferencia de las unitarias, aquí no interesa tanto aislar una clase como detectar errores de configuración, contratos incorrectos entre capas o fallos en la interacción entre módulos.

Las **pruebas end-to-end** reproducen un flujo completo desde la perspectiva del usuario o del consumidor de la API, recorriendo el sistema de extremo a extremo. En Gipsi este nivel se ha cubierto en esta fase mediante ejecuciones manuales guiadas, centradas en escenarios funcionales reales como autenticación, exploración y consulta de disponibilidad.

La diferencia principal entre estos tres niveles está en el alcance, la velocidad y el tipo de fallo que permiten detectar. Las unitarias son las más rápidas y precisas para aislar lógica interna; las de integración validan la cooperación entre piezas técnicas; y las end-to-end ofrecen la mayor confianza funcional, aunque son más costosas de ejecutar y mantener.

5.2. Pruebas Funcionales

Las pruebas funcionales verifican que el sistema se comporta correctamente desde la perspectiva del usuario, sin analizar el código interno.

5.2.1. Pruebas del Flujo de Autenticación

Tabla 5.1: Casos de prueba — Autenticación

ID	Caso	Resultado Esperado	Resultado
PF-A01	Login con credenciales correctas	HTTP 200, par de tokens en respuesta	✓
PF-A02	Login con contraseña incorrecta	HTTP 401, mensaje de error en español	✓
PF-A03	11 intentos de login seguidos	11ª petición devuelve HTTP 429	✓
PF-A04	Refresh con token válido	HTTP 200, nuevos tokens; el antiguo queda revocado	✓
PF-A05	Refresh con token ya rotado (simulación de robo)	HTTP 401, sesión invalidada	✓
PF-A06	Acceso a /me sin token	HTTP 401	✓
PF-A07	Acceso a /businessses/me/images con rol CUSTOMER	HTTP 403	✓
PF-A08	Registro con email ya existente	HTTP 409 Conflict	✓

5.2.2. Pruebas del Módulo de Exploración (Frontend)

Tabla 5.2: Casos de prueba — Exploración en frontend

ID	Caso	Resultado Esperado	Resultado
PF-E01	Abrir la pestaña Explorar con backend activo	Se cargan categorías y tarjetas de negocio	✓
PF-E02	Seleccionar categoría «Peluquería»	Lista filtrada solo con negocios de esa categoría	✓
PF-E03	Pull-to-refresh en la lista	La lista se recarga desde el servidor	✓
PF-E04	Tocar tarjeta de negocio	Navega a la pantalla de detalle	✓
PF-E05	Abrir con backend caído	Muestra estado de error con botón reintentar	✓
PF-E06	Filtro sin resultados	Muestra estado vacío con mensaje descriptivo	✓

5.2.3. Pruebas del Módulo de Disponibilidad

Tabla 5.3: Casos de prueba — Disponibilidad horaria

ID	Caso	Resultado Esperado	Resultado
PF-D01	Consultar disponibilidad día laborable	Lista de slots con correcta mezcla de disponibles e indisponibles	✓
PF-D02	Consultar disponibilidad día festivo (fuera de horario)	Todos los slots <code>OUT_OF_SCHEDULE</code>	✓
PF-D03	Slot con reserva existente	Slot marcado como <code>BOOKED</code>	✓
PF-D04	Consulta sin <code>workerId</code> en negocio multi-worker	Slots con <code>availableWorkerIds</code> rellenos	✓

5.3. Pruebas Unitarias

5.3.1. Pruebas del Backend con JUnit 5

Spring Boot incluye soporte para pruebas unitarias mediante JUnit 5 y Mockito. Los siguientes casos de prueba representan la batería básica para los componentes más críticos.

En el backend se han priorizado pruebas pequeñas y aisladas que validan reglas de negocio sensibles sin necesidad de levantar toda la aplicación ni depender de integraciones externas. Este enfoque permite comprobar con rapidez el comportamiento de componentes clave y detectar regresiones antes de que alcancen pruebas manuales o escenarios de integración.

Las pruebas incluidas en esta sección no se limitan a verificar que un método devuelve un dato esperado. También comprueban cómo responde el sistema ante tokens expirados,

manipulaciones maliciosas y conflictos reales de agenda. Por ello, constituyen una red de seguridad especialmente útil en dos áreas críticas del proyecto: la autenticación y el cálculo de disponibilidad.

JwtService

```
1 package com.gipsi.security;
2
3 import io.jsonwebtoken.JwtException;
4 import org.junit.jupiter.api.BeforeEach;
5 import org.junit.jupiter.api.Test;
6
7 import java.nio.charset.StandardCharsets;
8 import java.util.Base64;
9
10 import static org.assertj.core.api.Assertions.assertThat;
11 import static org.assertj.core.api.Assertions.assertThatThrownBy;
12
13 class JwtServiceTest {
14
15     private static final String SECRET =
16         Base64.getEncoder().encodeToString(
17
18             "testSecretKeyForJwtServiceTestsDeGipsiApp1234567890-HS512-safe-length"
19                 .getBytes(StandardCharsets.UTF_8)
20         );
21
22     private JwtService jwtService;
23
24     @BeforeEach
25     void setUp() {
26         jwtService = new JwtService(SECRET, 1_800_000L);
27         jwtService.init();
28     }
29
30     @Test
31     void generateToken_shouldContainUsername() {
32         String token = jwtService.generateToken("laurasanchez", "CUSTOMER", 1L);
33
34         assertThat(jwtService.extractUsername(token)).isEqualTo("laurasanchez");
35     }
36
37     @Test
38     void isValid_withExpiredToken_shouldReturnFalse() throws InterruptedException {
39         JwtService shortLivedJwtService = new JwtService(SECRET, 1L);
40         shortLivedJwtService.init();
41         String token = shortLivedJwtService.generateToken("test", "CUSTOMER", 1L);
42
43         Thread.sleep(25);
44
45         assertThat(shortLivedJwtService.isValid(token)).isFalse();
46     }
47
48     @Test
49     void extractUsername_withTamperedPayload_shouldThrowJwtException() {
50         String token = jwtService.generateToken("test", "CUSTOMER", 1L);
51         String[] parts = token.split("\\.");
52         String tamperedPayload = parts[1].substring(0, parts[1].length() - 1)
```

```

52         + (parts[1].endsWith("A") ? "B" : "A");
53     String tamperedToken = parts[0] + "." + tamperedPayload + "." + parts[2];
54
55     assertThatThrownBy(() -> jwtService.extractUsername(tamperedToken))
56         .assertInstanceOf(JwtException.class);
57 }
58 }

```

Listing 5.1: Pruebas unitarias de JwtService

Las pruebas de `JwtService` verifican el ciclo esencial de vida del token. La primera asegura que, tras generar un JWT, el nombre de usuario queda correctamente embebido y puede recuperarse después. Esto garantiza que el backend podrá reconstruir la identidad del actor autenticado a partir del token recibido en cada petición protegida.

La segunda fuerza la caducidad del token creando una instancia con un tiempo de vida mínimo. Con ello se comprueba que un token deja de considerarse válido una vez superada su ventana temporal, incluso aunque su formato siga siendo correcto. La tercera prueba modifica manualmente el *payload* del JWT para simular una manipulación externa; el resultado esperado es una excepción de tipo `JwtException`, lo que confirma que la firma deja de ser válida cuando se altera el contenido. En conjunto, estas verificaciones demuestran que el servicio no solo emite tokens, sino que también protege su integridad y aplica correctamente las restricciones de expiración.

AvailabilityService

```

1 package com.gipsi.service;
2
3 import com.gipsi.dto.availability.AvailabilityDtos.AvailabilityResponse;
4 import com.gipsi.dto.availability.AvailabilityDtos.AvailabilitySlot;
5 import com.gipsi.dto.availability.AvailabilityDtos.SlotUnavailableReason;
6 import com.gipsi.entity.Booking;
7 import com.gipsi.entity.Business;
8 import com.gipsi.entity.Schedule;
9 import com.gipsi.entity.ServiceOffered;
10 import com.gipsi.entity.Worker;
11 import com.gipsi.repository.AbsenceRepository;
12 import com.gipsi.repository.BookingRepository;
13 import com.gipsi.repository.ScheduleRepository;
14 import com.gipsi.repository.ServiceOfferedRepository;
15 import com.gipsi.repository.TimeOffRepository;
16 import com.gipsi.repository.WorkerRepository;
17 import org.junit.jupiter.api.BeforeEach;
18 import org.junit.jupiter.api.Test;
19 import org.junit.jupiter.api.extension.ExtendWith;
20 import org.mockito.Mock;
21 import org.mockito.junit.jupiter.MockitoExtension;
22
23 import java.time.LocalDate;
24 import java.time.LocalDateTime;
25 import java.time.LocalTime;
26 import java.util.List;
27 import java.util.Set;
28
29 import static org.assertj.core.api.Assertions.assertThat;
30 import static org.mockito.ArgumentMatchers.any;

```

```
31 import static org.mockito.ArgumentMatchers.anyCollection;
32 import static org.mockito.ArgumentMatchers.eq;
33 import static org.mockito.Mockito.when;
34
35 @ExtendWith(MockitoExtension.class)
36 class AvailabilityServiceTest {
37
38     @Mock
39     private ServiceOfferedRepository serviceRepository;
40
41     @Mock
42     private WorkerRepository workerRepository;
43
44     @Mock
45     private ScheduleRepository scheduleRepository;
46
47     @Mock
48     private TimeOffRepository timeOffRepository;
49
50     @Mock
51     private BookingRepository bookingRepository;
52
53     @Mock
54     private AbsenceRepository absenceRepository;
55
56     private AvailabilityService availabilityService;
57
58     @BeforeEach
59     void setUp() {
60         availabilityService = new AvailabilityService(
61             serviceRepository,
62             workerRepository,
63             scheduleRepository,
64             timeOffRepository,
65             bookingRepository,
66             absenceRepository
67         );
68     }
69
70     @Test
71     void getAvailability_whenWorkerHasTimeOff_allSlotsUnavailable() {
72         LocalDate date = LocalDate.now().plusDays(1);
73         Business business = buildBusiness(1L);
74         Worker worker = buildWorker(10L, business);
75         ServiceOffered service = buildService(100L, business, worker, 60);
76
77         when(serviceRepository.findById(service.getId())).thenReturn(java.util.Optional.of(service));
78
79         when(workerRepository.findById(worker.getId())).thenReturn(java.util.Optional.of(worker));
80         when(timeOffRepository.findOverlapping(eq(worker.getId()),
81             any(LocalDate.class), any(LocalDate.class)))
82             .thenReturn(List.of(new com.gipsi.entity.TimeOff()));
83
84         AvailabilityResponse response =
85             availabilityService.getAvailability(service.getId(), date, worker.getId());
86
87         assertThat(response.slots()).hasSize(96);
88     }
89 }
```

```

85     assertThat(response.slots()).allSatisfy(slot -> {
86         assertThat(slot.available()).isFalse();
87         assertThat(slot.reason()).isEqualTo(SlotUnavailableReason.TIME_OFF);
88     });
89 }
90
91 @Test
92 void getAvailability_whenSlotBooked_markedAsBooked() {
93     LocalDate date = LocalDate.now().plusDays(1);
94     Business business = buildBusiness(2L);
95     Worker worker = buildWorker(20L, business);
96     ServiceOffered service = buildService(200L, business, worker, 30);
97     Schedule schedule = buildSchedule(date, LocalTime.of(9, 0),
LocalTime.of(10, 0));
98
99     LocalDateTime bookingStart = LocalDateTime.of(date, LocalTime.of(9, 0));
100    LocalDateTime bookingEnd = LocalDateTime.of(date, LocalTime.of(9, 30));
101
102    when(serviceRepository.findById(service.getId())).thenReturn(java.util.Optional.of(service));
103
104    when(workerRepository.findById(worker.getId())).thenReturn(java.util.Optional.of(worker));
105    when(timeOffRepository.findOverlapping(eq(worker.getId()),
any(LocalDate.class), any(LocalDate.class)))
        .thenReturn(List.of());
106    when(absenceRepository.findApprovedOverlapping(eq(worker.getId()),
any(LocalDate.class), any(LocalDate.class)))
        .thenReturn(List.of());
107    when(scheduleRepository.findByActorIdAndDayOfWeek(worker.getId(),
date.getDayOfWeek()))
        .thenReturn(List.of(schedule));
108    when(bookingRepository.findOverlappingForWorker(
eq(worker.getId()),
any(LocalDateTime.class),
any(LocalDateTime.class),
anyCollection()))
        .thenAnswer(invocation -> {
109        LocalDateTime slotStart = invocation.getArgument(1);
110        LocalDateTime slotEnd = invocation.getArgument(2);
111        boolean overlaps = slotStart.isBefore(bookingEnd) &&
slotEnd.isAfter(bookingStart);
112        return overlaps ? List.of(new Booking()) : List.of();
113    });
114
115    AvailabilityResponse response =
availabilityService.getAvailability(service.getId(), date, worker.getId());
116
117    AvailabilitySlot nineOClock = response.slots().stream()
        .filter(slot -> slot.start().equals(LocalTime.of(9, 0)))
        .findFirst()
        .orElseThrow();
118
119    assertThat(nineOClock.available()).isFalse();
120    assertThat(nineOClock.reason()).isEqualTo(SlotUnavailableReason.BOOKED);
121    assertThat(nineOClock.end()).isEqualTo(LocalTime.of(9, 30));
122 }
123
124 private Business buildBusiness(Long id) {

```

```
135     Business business = new Business();
136     business.setId(id);
137     business.setAutonomous(false);
138     business.setAvailabilityHorizonDays(60);
139     return business;
140 }
141
142 private Worker buildWorker(Long id, Business business) {
143     Worker worker = new Worker();
144     worker.setId(id);
145     worker.setBusiness(business);
146     worker.setAvailable(true);
147     worker.setAllowOvertime(false);
148     return worker;
149 }
150
151 private ServiceOffered buildService(Long id, Business business, Worker worker,
152 int duration) {
153     ServiceOffered service = new ServiceOffered();
154     service.setId(id);
155     service.setBusiness(business);
156     service.setDuration(duration);
157     service.setActive(true);
158     service.setWorkers(Set.of(worker));
159     return service;
160 }
161
162 private Schedule buildSchedule(LocalDate date, LocalTime start, LocalTime end) {
163     Schedule schedule = new Schedule();
164     schedule.setDayOfWeek(date.getDayOfWeek());
165     schedule.setStartTime(start);
166     schedule.setEndTime(end);
167     return schedule;
168 }
```

Listing 5.2: Pruebas unitarias del servicio de disponibilidad

Las pruebas de `AvailabilityService` se centran en validar la lógica que traduce horarios, reservas y ausencias en tramos de disponibilidad consumibles por el cliente. Para ello se aíslan los repositorios con Mockito y se controla el estado de entrada de cada dependencia, de forma que el resultado dependa únicamente de las reglas de negocio del servicio.

La primera prueba simula que el trabajador tiene un periodo de `TimeOff` solapado con la fecha consultada. El resultado esperado es que los 96 slots de quince minutos del día se marquen como no disponibles y que todos informen de la razón `TIME_OFF`. De este modo se verifica que la indisponibilidad global del trabajador prevalece sobre cualquier otra posible condición.

La segunda prueba reproduce un escenario más preciso: el trabajador sí tiene horario, no presenta ausencias aprobadas y existe una reserva previa exactamente en la franja de las 09:00 a las 09:30. La comprobación final confirma que ese slot concreto aparece como no disponible, que la causa informada es `BOOKED` y que la duración del tramo coincide con la del servicio solicitado. Esto valida que el algoritmo detecta solapes reales con reservas existentes y devuelve al frontend una representación fiel de qué huecos

pueden ofrecerse y cuáles no.

5.4. Evaluación del Sistema

5.4.1. Pruebas de Rendimiento Manual

Se realizaron pruebas manuales de rendimiento midiendo el tiempo de respuesta de los endpoints más frecuentes con la base de datos con datos de prueba cargados por `DataInitializer`:

Tabla 5.4: Tiempos de respuesta medidos en entorno local

Endpoint	Tiempo medio (ms)	Requisito (RNF-1)
GET /businesses?page=0&size=20	87	<500 ms ✓
GET /businesses/{id} (con reviews)	134	<500 ms ✓
GET /services/{id}/availability	203	<500 ms ✓
POST /auth/login	312	<500 ms ✓
POST /auth/refresh	45	<500 ms ✓

El tiempo de login es el más elevado debido al costo computacional del hash bcrypt (factor de trabajo 10 por defecto), lo cual es deliberado: un atacante que prueba miles de contraseñas por segundo experimenta el mismo retardo.

5.5. Capturas de las Pruebas

5.5.1. Pruebas del flujo de autenticación

Procedimiento manual para obtener las capturas

Para documentar las pruebas del flujo de autenticación se ejecuta cada caso con el backend iniciado en perfil `dev`, usando los usuarios cargados por `DataInitializer`.

1. Iniciar el backend con el perfil `dev` y comprobar que la API responde en `http://localhost:8080`.
2. Abrir Postman, Insomnia o la terminal con `curl`. Crear una petición por cada caso de la Tabla 5.1.
3. Ejecutar la prueba, verificar que el estado HTTP coincide con el resultado esperado y realizar una captura de pantalla.

Tabla 5.5: Guía de ejecución manual para capturas del flujo de autenticación

ID	Preparación y acción	Captura a guardar
PF-A01	Enviar POST /auth/login con laurasanchez/password123. Comprobar 200 OK y presencia de accessToken y refreshToken.	pf-a01-login-correcto.png
PF-A02	Enviar POST /auth/login con laurasanchez y una contraseña incorrecta. Comprobar 401 Unauthorized.	pf-a02-login-error.png
PF-A03	Repetir 11 veces el login incorrecto desde el mismo cliente/IP. La captura debe mostrar la undécima respuesta con 429 Too Many Requests.	pf-a03-rate-limit-login.png
PF-A04	Hacer login correcto, copiar el refreshToken y enviarlo a POST /auth/refresh. Comprobar 200 OK y nuevos tokens.	pf-a04-refresh-valido.png
PF-A05	Reutilizar el refreshToken antiguo del caso PF-A04. Comprobar 403 Forbidden y que la sesión queda invalidada.	pf-a05-refresh-reutilizado.png
PF-A06	Enviar GET /me sin cabecera Authorization. Comprobar 401 Unauthorized.	pf-a06-me-sin-token.png
PF-A07	Hacer login como laurasanchez, usar su accessToken en POST /businessses/me/images/profile y comprobar 403 Forbidden.	pf-a07-rol-customer.png
PF-A08	Enviar POST /auth/register/customer con un email ya existente. Comprobar 409 Conflict.	pf-a08-email-existente.png

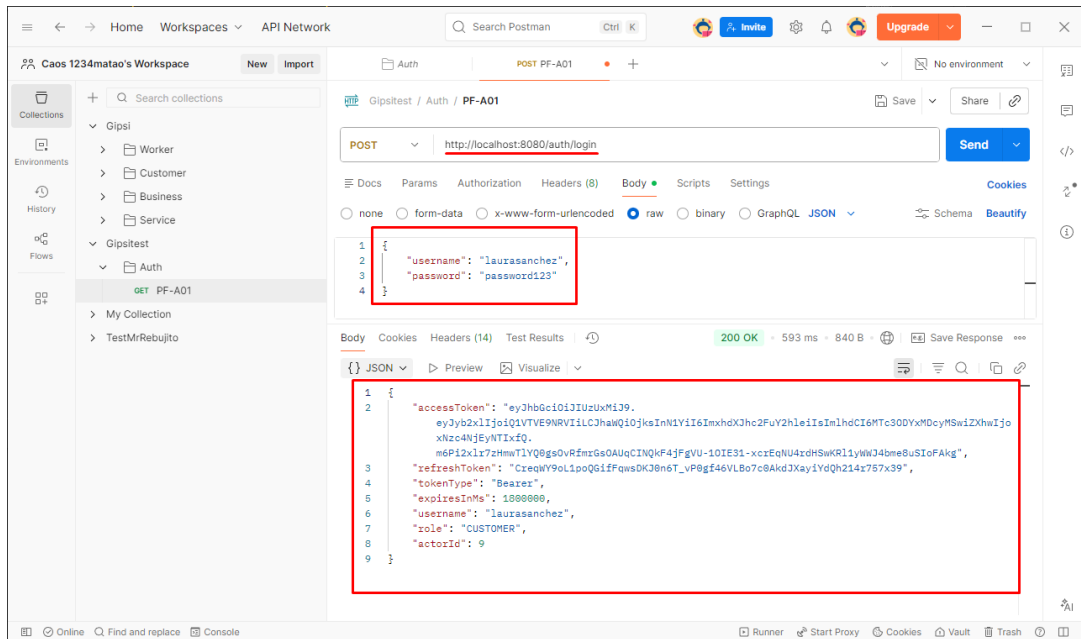


Figura 5.2: PF-A01 — Login con credenciales correctas

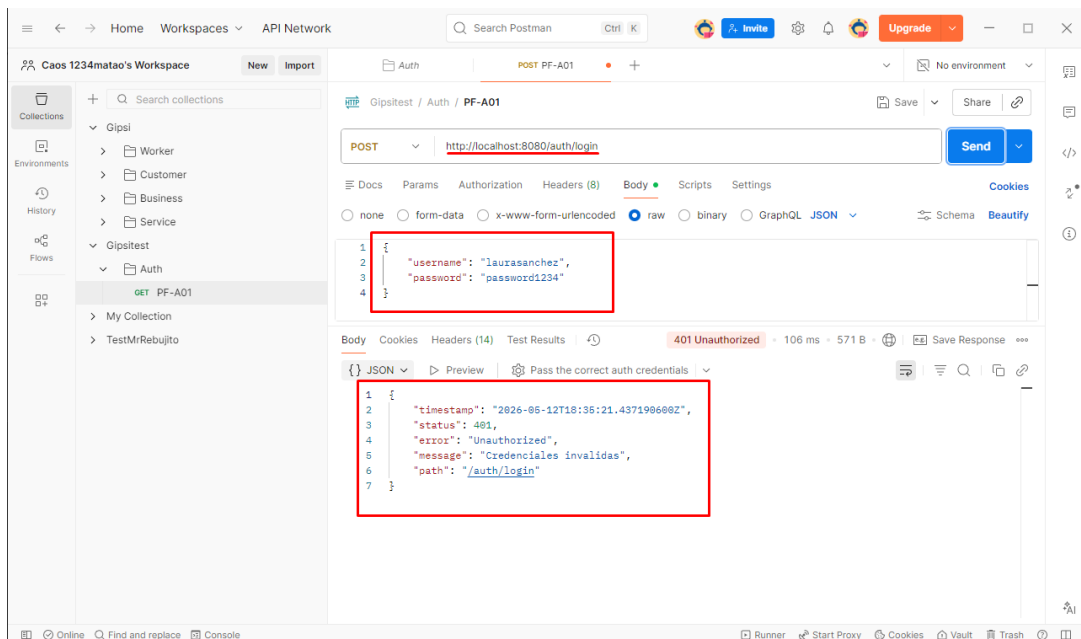


Figura 5.3: PF-A02 — Login con contraseña incorrecta

Captura pendiente

capturas_pruebas/pf-a03-rate-limit-login.png

Figura 5.4: PF-A03 — Rate limiting tras 11 intentos de login

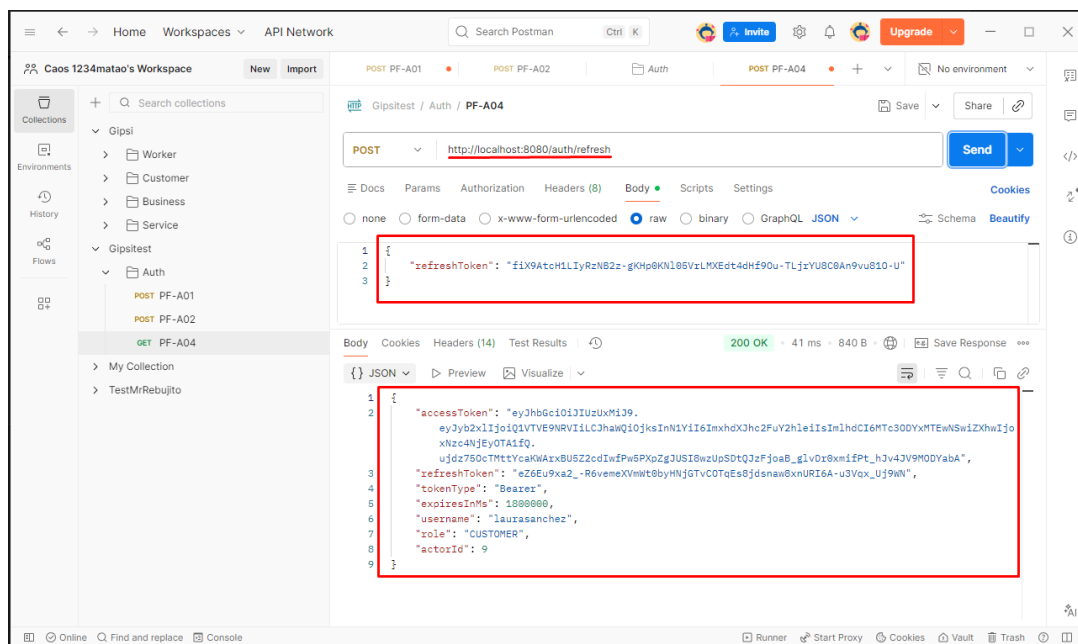


Figura 5.5: PF-A04 — Refresh con token válido

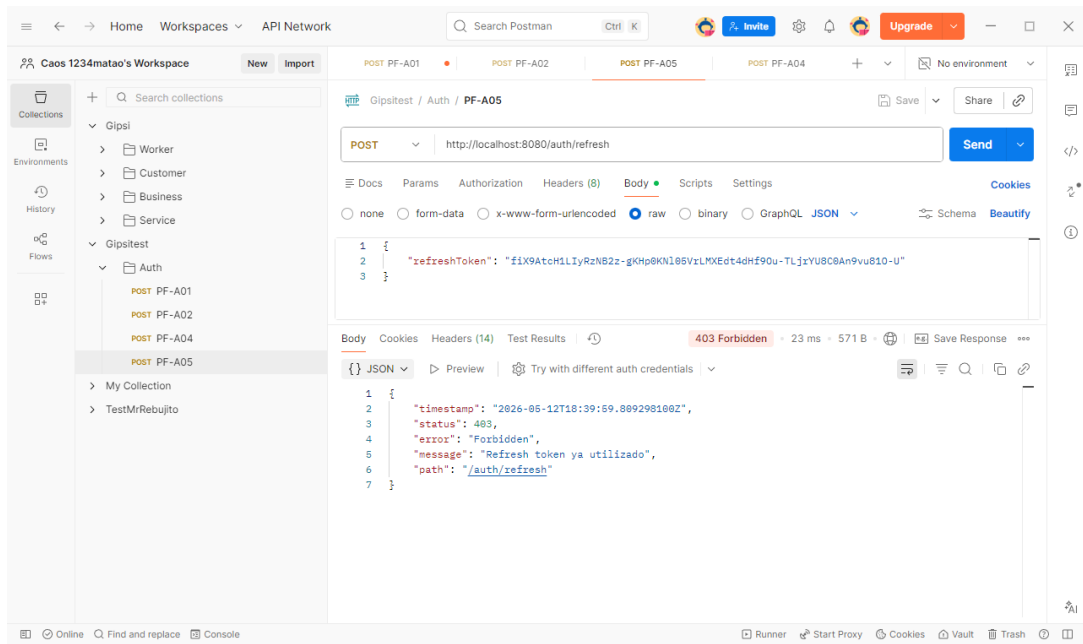


Figura 5.6: PF-A05 — Refresh con token ya rotado

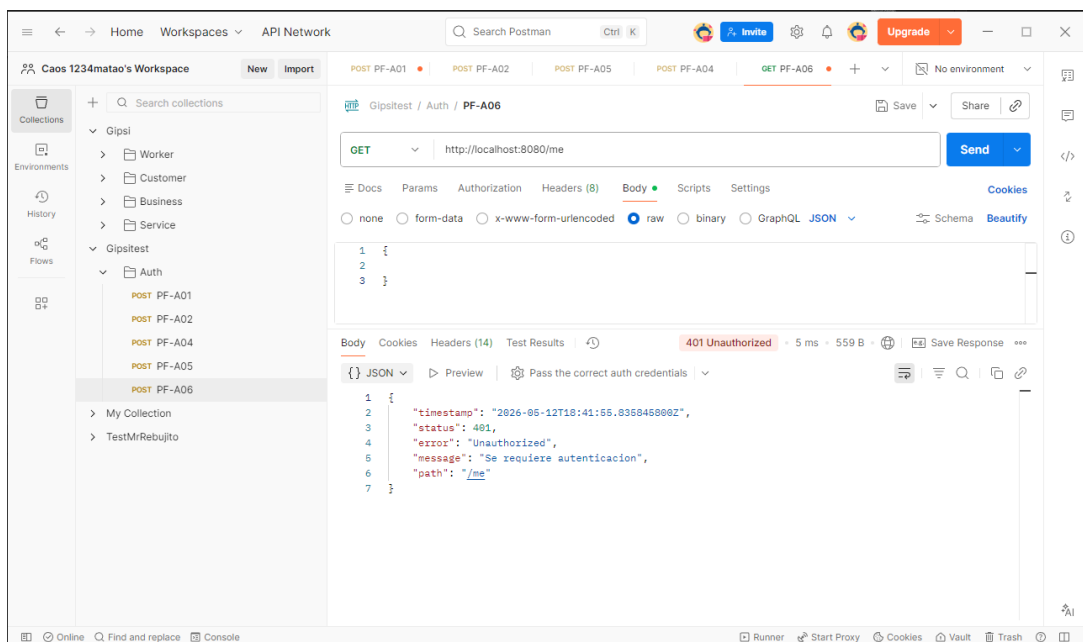


Figura 5.7: PF-A06 — Acceso al endpoint /me sin token

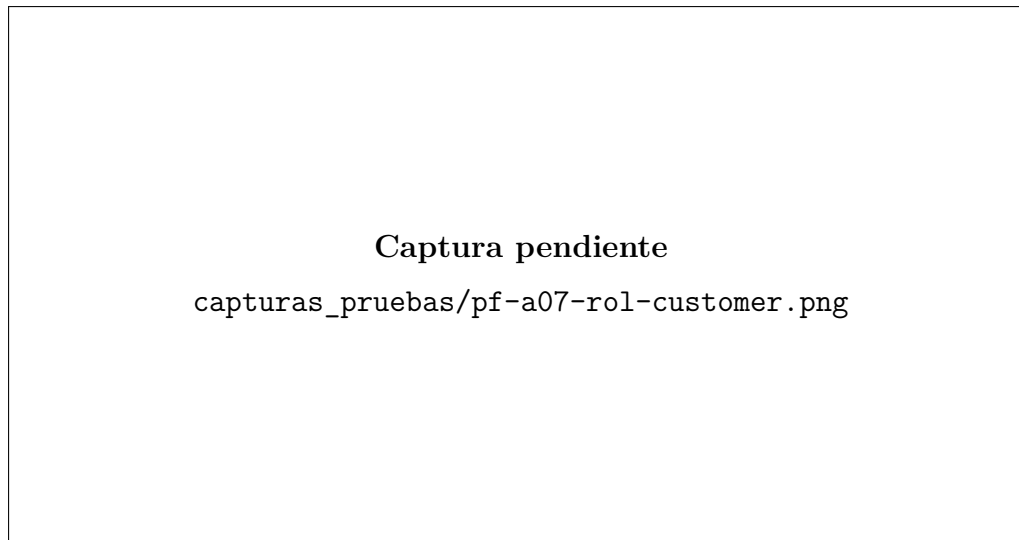


Figura 5.8: PF-A07 — Acceso a recurso de negocio con rol CUSTOMER

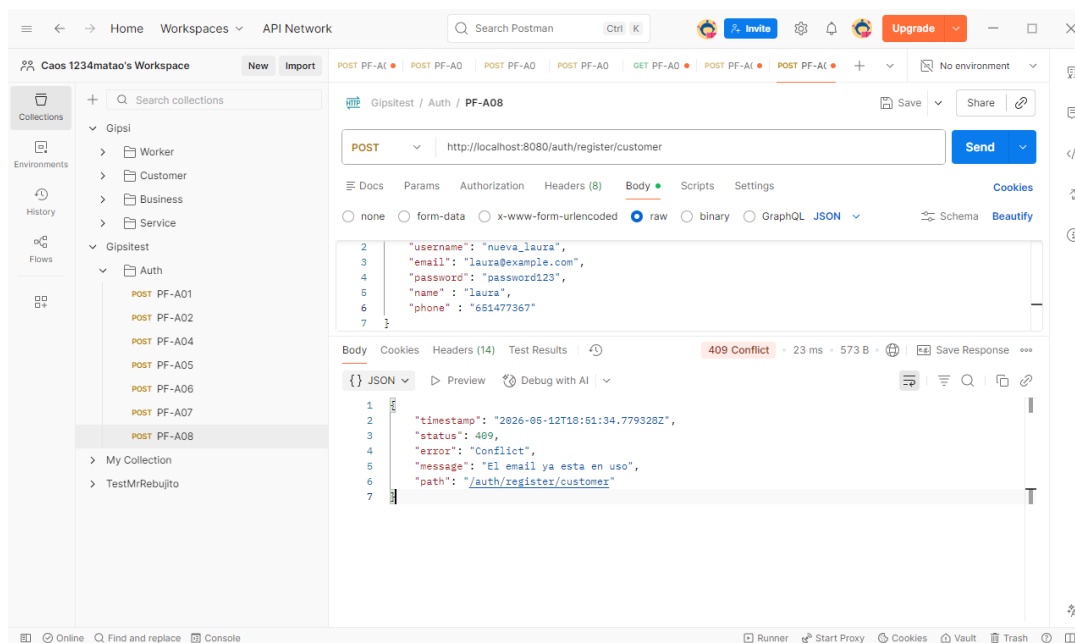


Figura 5.9: PF-A08 — Registro con email ya existente

5.5.2. Pruebas del módulo de exploración

Procedimiento manual para obtener las capturas

Para documentar las pruebas de exploración se ejecuta la app Flutter conectada al backend local con datos de ejemplo cargados por `DataInitializer`. Las capturas se realizan desde el emulador Android, simulador iOS o dispositivo físico mientras se navega por la pestaña **Explorar**.

1. Iniciar el backend con el perfil `dev` y comprobar que la API responde en `http://localhost:8080`.
2. Lanzar el frontend con `flutter run` y verificar que la app carga la pestaña **Explorar** consumiendo `GET /categories` y `GET /businesses` con paginación y

orden ascendente.

3. Ejecutar cada caso de la Tabla 5.6, verificando el comportamiento visual esperado antes de realizar la captura.
4. Guardar cada imagen con el nombre indicado para mantener trazabilidad directa entre el caso de prueba y la evidencia visual.

Tabla 5.6: Guía de ejecución manual para capturas del módulo de exploración

ID	Preparación y acción	Captura a guardar
PF-E01	Abrir la pestaña Explorar con backend activo y esperar a que terminen la carga inicial de categorías y tarjetas. Comprobar que se muestran negocios reales.	pf-e01-explorar-inicial.png
PF-E02	Seleccionar la categoría Peluquería y comprobar que el listado queda filtrado mostrando solo negocios de esa categoría.	pf-e02-explorar-peluqueria.png
PF-E03	Realizar gesto de <i>pull-to-refresh</i> sobre la lista y capturar el estado de recarga mientras se solicitan de nuevo los datos al servidor.	pf-e03-explorar-refresh.png
PF-E04	Tocar una tarjeta de negocio desde Explorar y comprobar que la navegación lleva a la vista de detalle del negocio seleccionado.	pf-e04-explorar-detalle.png
PF-E05	Repetir la apertura de Explorar con el backend detenido o inaccesible para comprobar que aparece el estado de error con botón Reintentar .	pf-e05-explorar-error.png
PF-E06	Aplicar un filtro o dataset de prueba sin resultados y comprobar que se muestra el estado vacío con mensaje descriptivo.	pf-e06-explorar-vacio.png

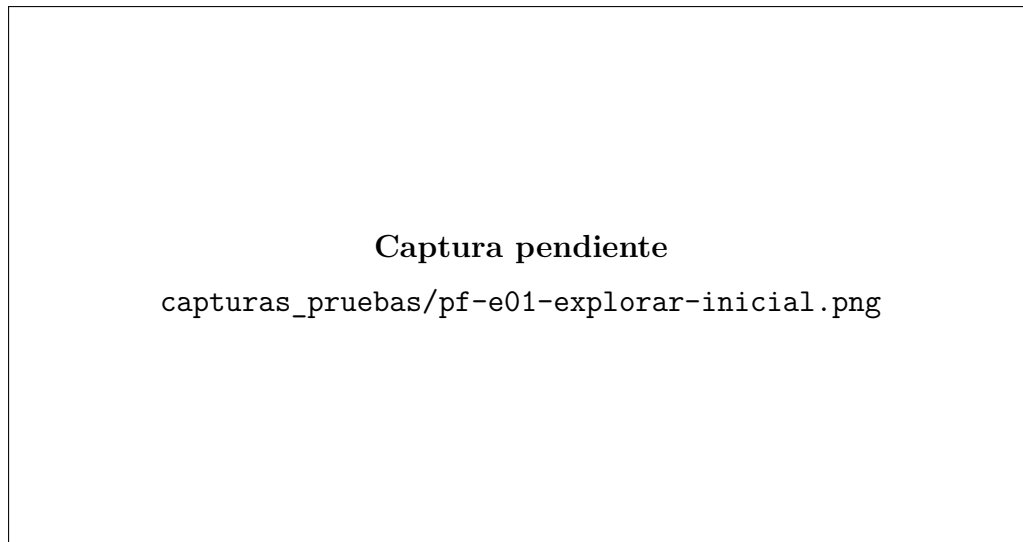


Figura 5.10: PF-E01 — Carga inicial de la pestaña Explorar

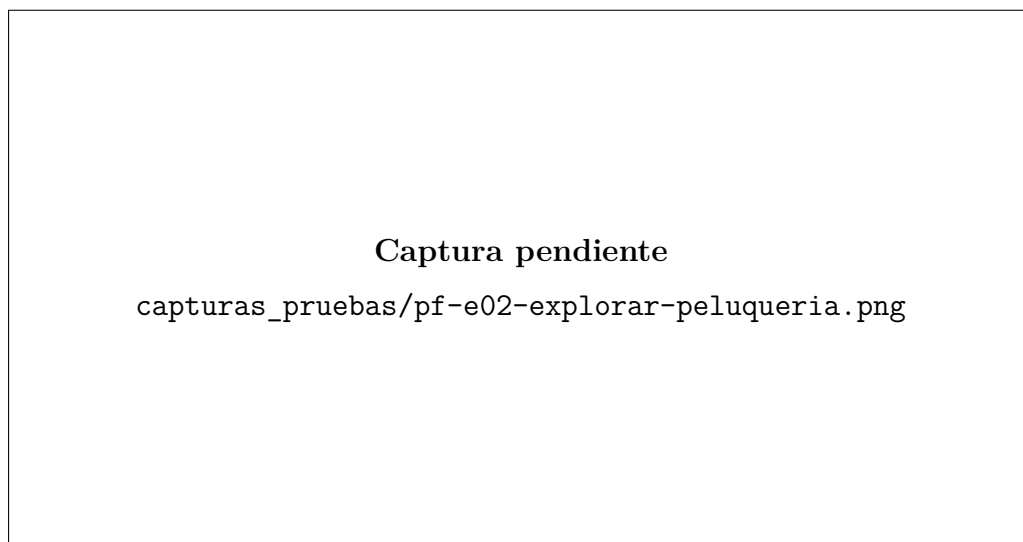


Figura 5.11: PF-E02 — Filtrado por categoría Peluquería

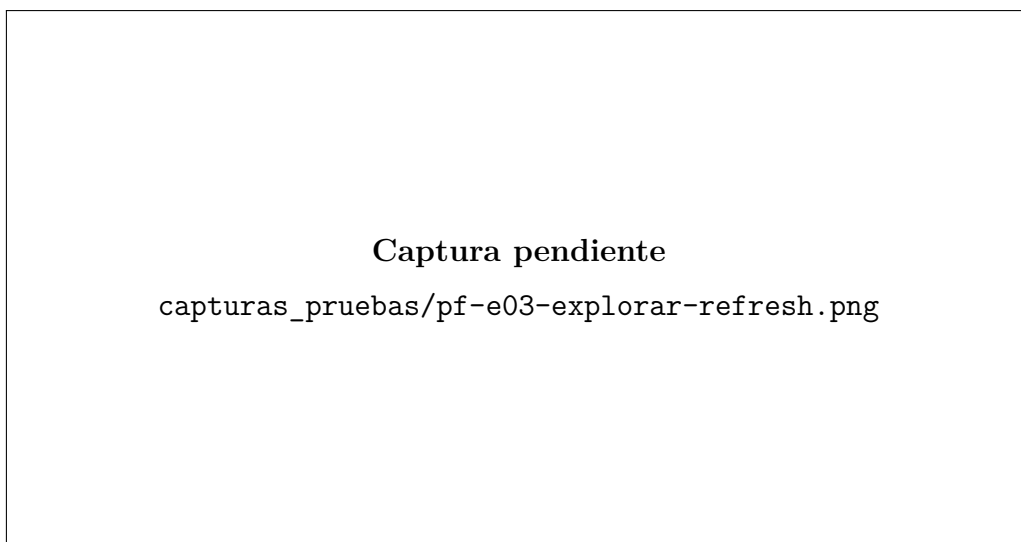


Figura 5.12: PF-E03 — Recarga manual con pull-to-refresh

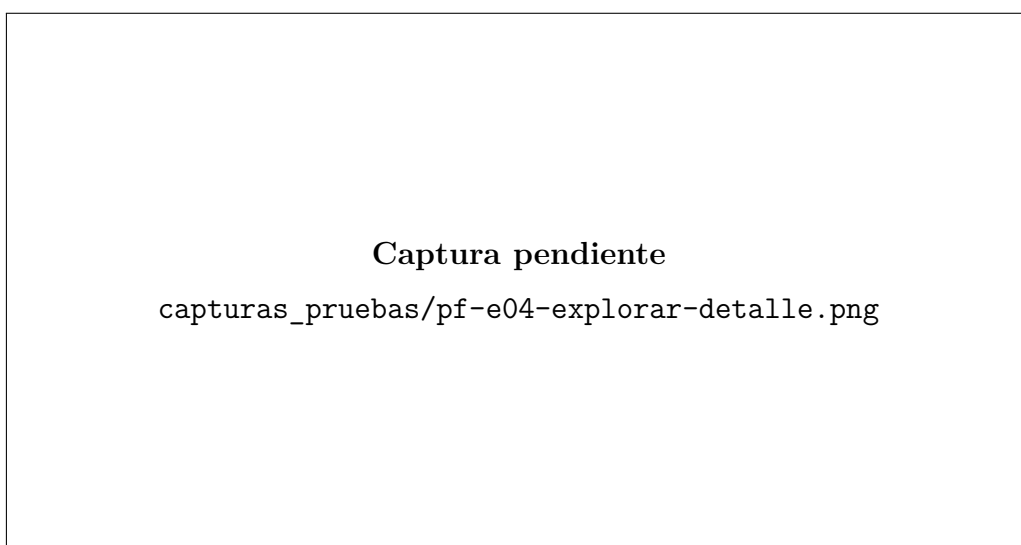


Figura 5.13: PF-E04 — Navegación al detalle de negocio

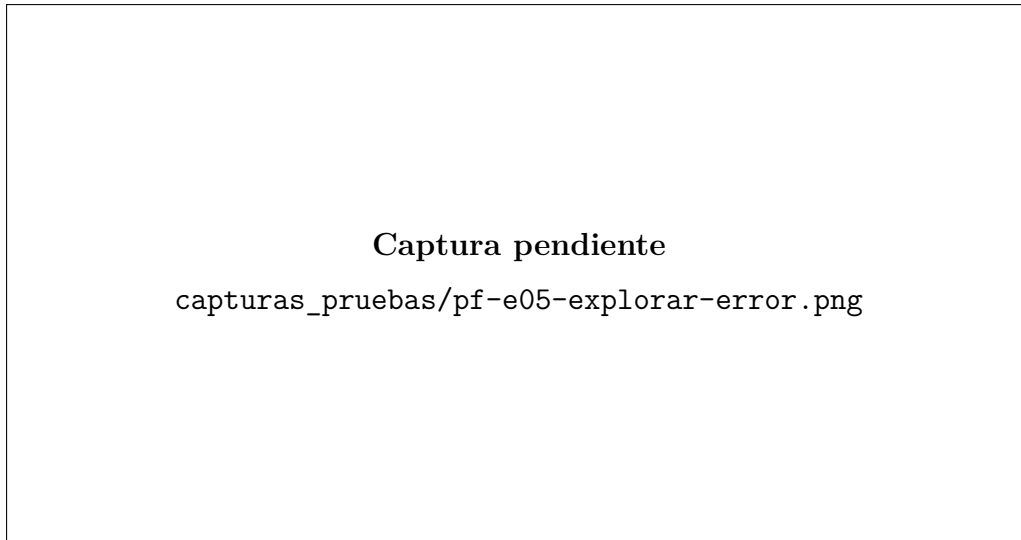


Figura 5.14: PF-E05 — Estado de error con backend caído

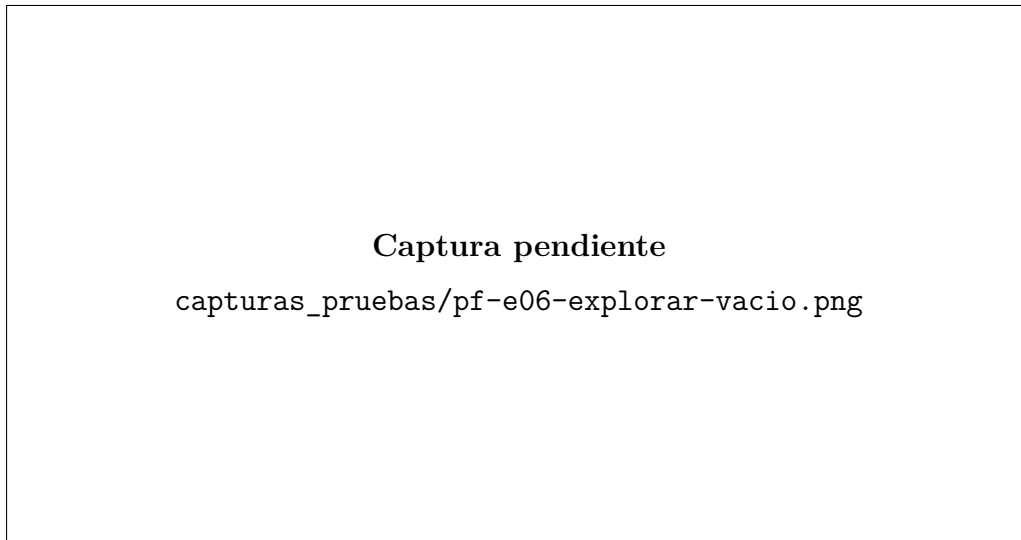


Figura 5.15: PF-E06 — Estado vacío sin resultados

5.5.3. Pruebas del módulo de disponibilidad

Procedimiento manual para obtener las capturas

Para documentar las pruebas de disponibilidad se consulta manualmente el endpoint **GET /services/{id}/availability** desde Postman, Insomnia o Swagger UI, utilizando el backend local con el perfil **dev**. Este enfoque permite inspeccionar directamente los campos **slots**, **reason** y **availableWorkerIds** que constituyen la evidencia funcional del cálculo horario.

1. Iniciar el backend con el perfil **dev** y abrir Swagger UI o Postman apuntando a **http://localhost:8080**.
2. Localizar un **serviceId** de prueba y, cuando aplique, un **workerId**, a partir de los datos expuestos por la API o de los negocios de ejemplo cargados por **DataInitializer**.

3. Ejecutar una petición por cada caso de la Tabla 5.7, comprobando que la respuesta JSON refleja el escenario esperado.
4. Guardar la captura de la respuesta con el nombre indicado para que la evidencia quede alineada con el ID del caso de prueba.

Tabla 5.7: Guía de ejecución manual para capturas del módulo de disponibilidad

ID	Preparación y acción	Captura a guardar
PF-D01	Consultar GET /services/{id}/availability indicando date=YYYY-MM-DD para un día laborable con horario activo. Comprobar mezcla de slots disponibles e indisponibles.	pf-d01-disponibilidad-laborable.png
PF-D02	Consultar la disponibilidad para una fecha fuera de horario o festiva y comprobar que todos los slots devueltos aparecen con motivo OUT_OF_SCHEDULE.	pf-d02-disponibilidad-fuera-horario.png
PF-D03	Consultar una fecha con una reserva previa y comprobar que el slot afectado se devuelve con motivo BOOKED.	pf-d03-disponibilidad-booked.png
PF-D04	Repetir la consulta omitiendo workerId en un negocio con varios trabajadores y comprobar que la respuesta incluye availableWorkerIds.	pf-d04-disponibilidad-multiworker.png

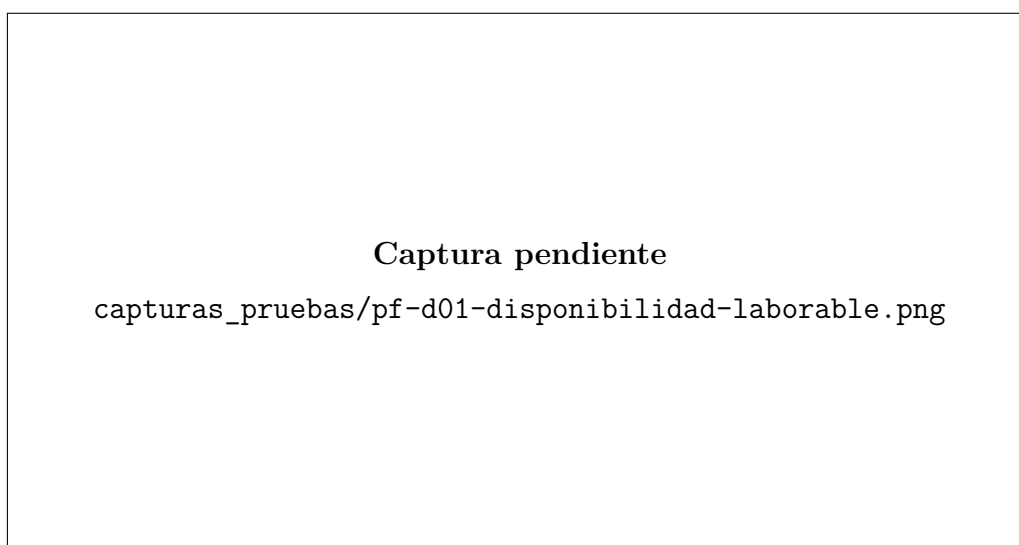


Figura 5.16: PF-D01 — Disponibilidad en día laborable

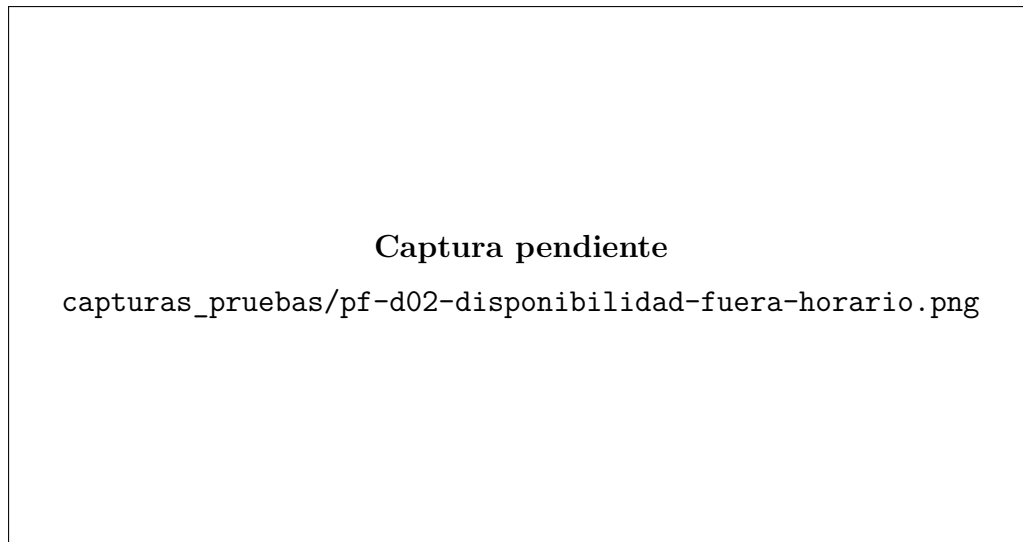


Figura 5.17: PF-D02 — Disponibilidad fuera de horario

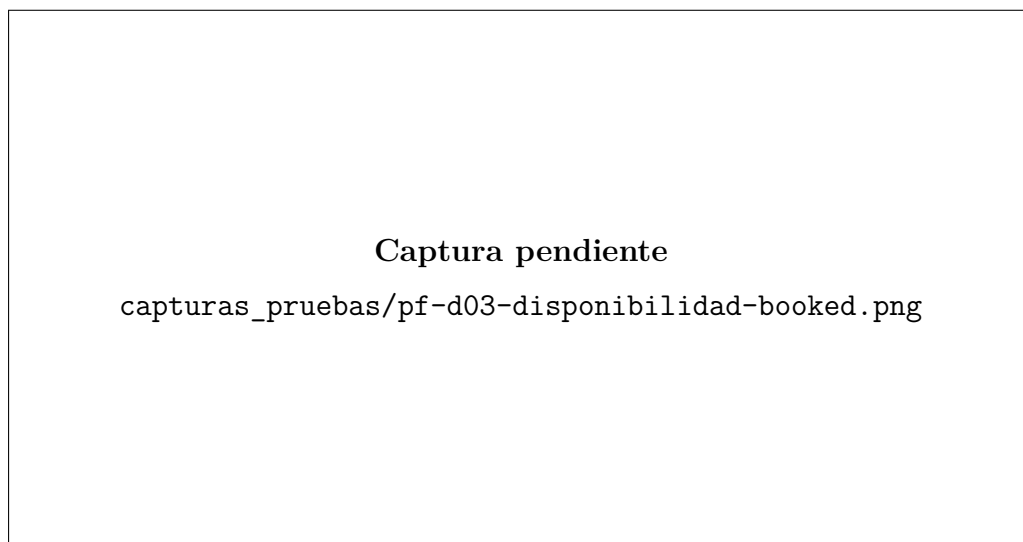


Figura 5.18: PF-D03 — Slot marcado como BOOKED

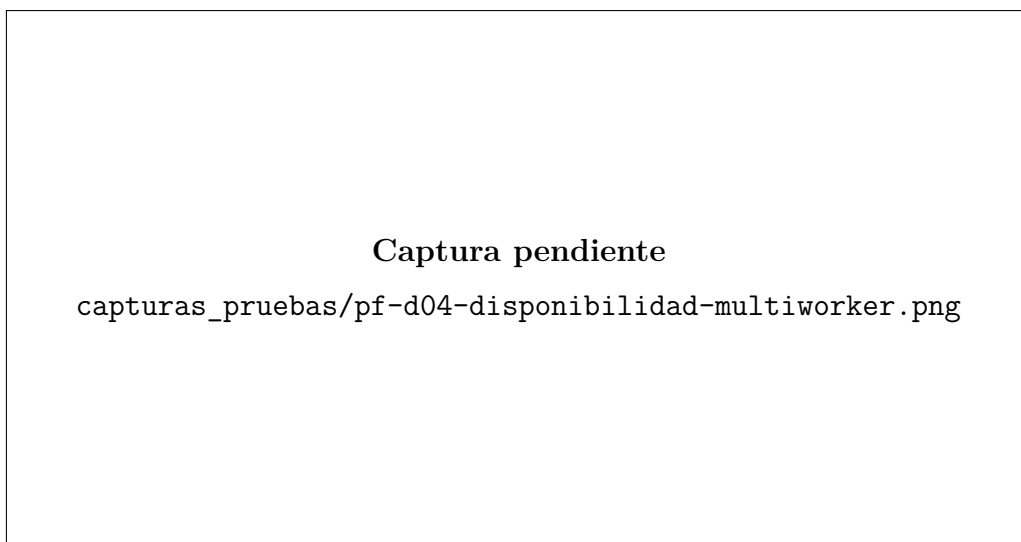


Figura 5.19: PF-D04 — Disponibilidad agregada en negocio multi-worker

5.5.4. Capturas de pruebas unitarias del backend

Procedimiento seguido para obtener las capturas

Las evidencias visuales del punto 5.3.1 se obtuvieron ejecutando las pruebas unitarias del backend desde el proyecto `repos/Gipsi-API`, utilizando el runner del entorno de desarrollo para dejar visibles todos los métodos de cada clase. Como en esta documentación se incluyó una captura por clase de prueba, se ejecutó cada clase completa y se guardó la imagen una vez que todos sus métodos aparecieron en estado correcto.

1. Se abrió el proyecto backend y se accedió a las clases `JwtServiceTest` y `AvailabilityServiceTest`.
2. Se ejecutó cada clase completa, sin lanzar los métodos `@Test` por separado.
3. Se comprobó que todos los métodos aparecían en verde o como finalizados correctamente dentro del runner.
4. Se guardó una captura por clase con el nombre indicado para mantener la correspondencia directa entre la evidencia y la prueba documentada en el punto 5.3.1.

Tabla 5.8: Resumen del procedimiento seguido para las capturas de las pruebas unitarias del backend

ID	Clase y evidencia	Captura guardada
PU-B01	Se ejecutó la clase completa <code>JwtServiceTest</code> . En la captura aparecieron en verde los métodos <code>generateToken_shouldContainUsername</code> , <code>isValid_withExpiredToken_shouldReturnFalse</code> y <code>extractUsername_withTamperedPayload_shouldThrowJwtException</code> .	<code>pu-b01-jwt-service-completo.png</code>
PU-B02	Se ejecutó la clase completa <code>AvailabilityServiceTest</code> . En la captura aparecieron en verde los métodos <code>getAvailability_whenWorkerHasTimeOff_allSlotsUnavailable</code> y <code>getAvailability_whenSlotBooked_markedAsBooked</code> .	<code>pu-b02-availability-service-completo.png</code>

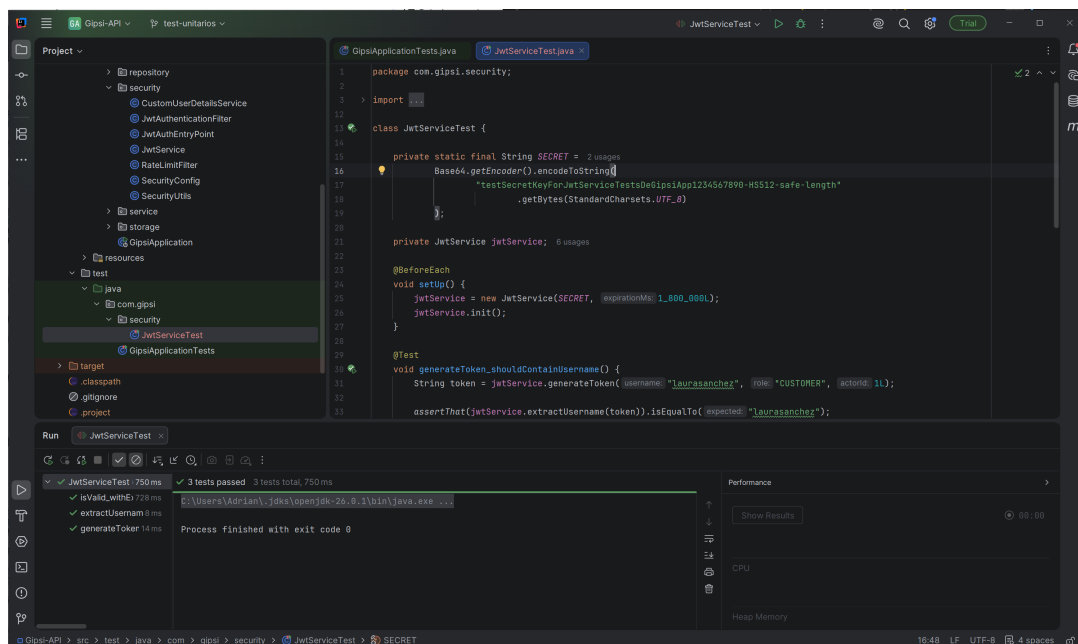


Figura 5.20: PU-B01 — Ejecucion completa de JwtServiceTest

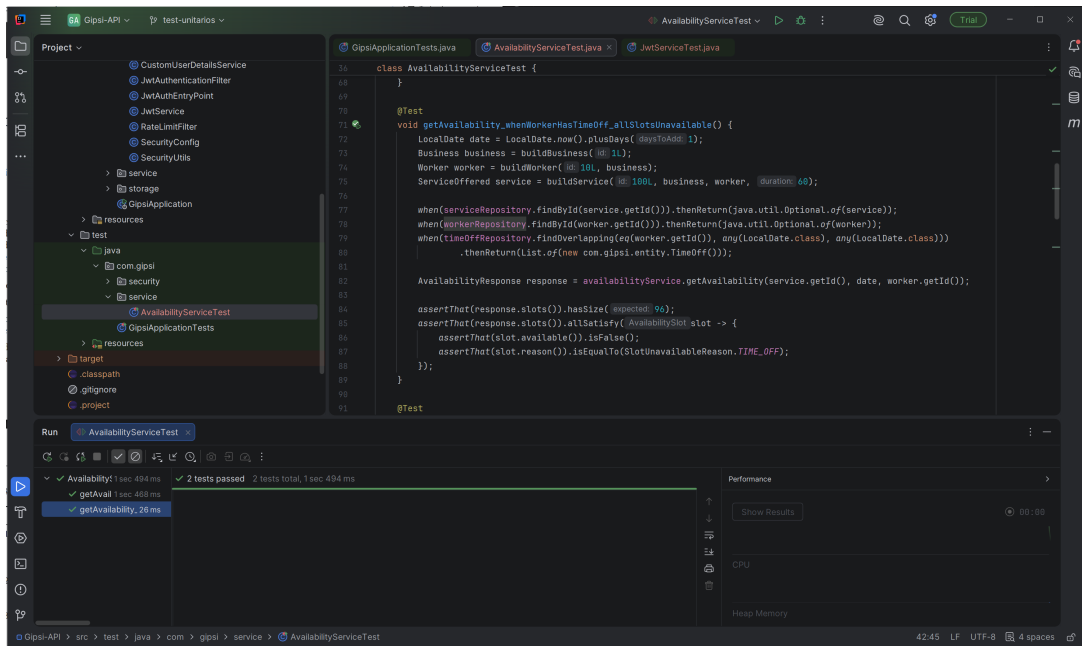


Figura 5.21: PU-B02 — Ejecucion completa de AvailabilityServiceTest

Capítulo 6

Despliegue

6.1. Requisitos de Entorno

El despliegue de Gipsi se divide en dos piezas principales: una API REST Spring Boot que mantiene el estado del sistema y una aplicación Flutter que consume dicha API. Esta separación permite publicar el backend en un servidor o contenedor propio, mientras que el frontend se distribuye como aplicación móvil compilada para Android o iOS. Por ello, antes de poner el sistema en producción es importante fijar tres aspectos: el entorno donde se ejecutará la API, la base de datos PostgreSQL persistente y la URL pública que utilizarán las aplicaciones cliente.

En desarrollo se admiten valores por defecto cómodos, como CORS abierto o una base de datos local. En producción, sin embargo, el despliegue debe realizarse con el perfil **prod**, secretos externos al repositorio, HTTPS y una lista cerrada de orígenes permitidos. Esta diferencia evita que una configuración pensada para pruebas acabe expuesta en un entorno real.

6.1.1. Backend

Tabla 6.1: Requisitos de entorno del backend

Componente	Versión / Requisito
Java	21 (LTS)
Maven	3.9+ (o usar el wrapper <code>mvnw</code> incluido)
PostgreSQL	15+
Firebase Admin credentials	JSON de cuenta de servicio (opcional para push)
RAM mínima	512 MB (JVM + Spring Boot)

Además de las versiones indicadas, el backend necesita acceso de red estable a PostgreSQL y permisos de escritura en el directorio configurado para imágenes. La carpeta de almacenamiento no debe residir dentro del artefacto de la aplicación, ya que las subidas de usuarios tienen que sobrevivir a reinicios, actualizaciones del JAR o recreaciones de contenedores. Si se habilitan las notificaciones push, el JSON de Firebase debe montarse como archivo externo y con permisos restrictivos.

6.1.2. Frontend

Tabla 6.2: Requisitos de entorno del frontend

Componente	Versión / Requisito
Flutter SDK	3.24.0+ (Dart 3.5+)
Android Studio	Hedgehog+ (para Android)
Xcode	15+ (para iOS, solo en macOS)
Melos	6.2.0+ (<code>dart pub global activate melos</code>)

6.1.3. Servicios Externos y Red

El entorno de producción también depende de varios elementos externos al código fuente. La siguiente tabla resume los más relevantes:

Tabla 6.3: Servicios externos necesarios para producción

Elemento	Consideración de despliegue
Dominio de la API	Debe resolver a la instancia backend, por ejemplo <code>api.gipsi.com</code>
HTTPS	Recomendado mediante proxy inverso como Nginx, Traefik o Caddy
CORS	Debe limitarse al dominio real de la aplicación cliente
PostgreSQL	Debe contar con volumen persistente y copias de seguridad periódicas
Almacenamiento	Directorio o volumen persistente para imágenes subidas por usuarios
Firebase	Opcional; si no se configura, la API arranca sin notificaciones push

La API puede escuchar internamente en el puerto 8080, pero en un despliegue público lo recomendable es situarla detrás de un proxy HTTPS. De esta forma el certificado TLS, la redirección de HTTP a HTTPS y posibles límites de tamaño de subida se gestionan en una capa específica, mientras que Spring Boot conserva una configuración sencilla y portable.

6.2. Configuración del Backend

6.2.1. Variables de Entorno

El backend se configura íntegramente mediante variables de entorno, sin credenciales en el repositorio:

```
# Base de datos
DB_URL=jdbc:postgresql://localhost:5432/gipsi
DB_USER=postgres
DB_PASSWORD=your_secure_password
```

```
# JWT - usar un secreto real de al menos 64 bytes en base64
JWT_SECRET=your_base64_encoded_secret_of_at_least_64_bytes
JWT_EXPIRATION_MS=1800000      # 30 minutos
REFRESH_EXPIRATION_DAYS=30

# URL base (para URLs absolutas de imágenes en respuestas)
GIPSI_BASE_URL=https://api.gipsi.com

# CORS - nunca usar * en producción
CORS_ALLOWED_ORIGINS=https://app.gipsi.com

# Almacenamiento de imágenes
STORAGE_TYPE=local             # 'local' o 's3' (futuro)
STORAGE_PATH=/var/gipsi/storage

# Firebase (dejar vacío para deshabilitar push)
FIREBASE_CREDENTIALS_PATH=/etc/gipsi/firebase-credentials.json

# Perfil Spring
SPRING_PROFILES_ACTIVE=prod
```

Listing 6.1: Variables de entorno del backend

Estas variables cubren las decisiones sensibles del despliegue. La URL de base de datos cambia entre desarrollo y producción; el secreto JWT debe generarse con suficiente entropía y no reutilizar el valor de desarrollo; y `GIPSI_BASE_URL` tiene que coincidir con la URL pública de la API, porque se utiliza para construir rutas absolutas de imágenes en las respuestas. También es importante que `CORS_ALLOWED_ORIGINS` contenga únicamente los dominios previstos. Usar `*` facilita las pruebas locales, pero no debería mantenerse en producción.

El valor `DATA_INITIALIZE` no aparece activado en la configuración de producción. Esta decisión es intencionada: los datos de prueba son útiles para desarrollo, pero en un servidor real podrían mezclar usuarios ficticios con datos legítimos. Si se necesita cargar información inicial en producción, debería hacerse mediante migraciones controladas o scripts administrativos separados.

6.2.2. Proceso de Arranque

```
# 1. Crear la base de datos
psql -U postgres -c "CREATE DATABASE gipsi;"

# 2. Arrancar (Flyway aplica migraciones automáticamente)
./mvnw spring-boot:run

# --- O bien, para producción ---

# 3. Compilar el JAR
./mvnw clean package -DskipTests

# 4. Ejecutar el JAR
java -jar target/gipsi-backend-0.0.1-SNAPSHOT.jar

# Verificar: Swagger disponible en http://localhost:8080/swagger-ui.html
```

Listing 6.2: Comandos de arranque del backend

Para un entorno productivo conviene crear un usuario de base de datos propio en lugar de usar el superusuario `postgres`. El backend solo necesita permisos sobre la base de datos de la aplicación, por lo que aislar el usuario reduce el impacto de una credencial comprometida.

```
# Crear usuario y base de datos de aplicación
psql -U postgres -c "CREATE USER gipsi_user WITH PASSWORD 'change_me';"
psql -U postgres -c "CREATE DATABASE gipsi OWNER gipsi_user;"

# Comprobar conectividad antes de arrancar la API
psql "postgresql://gipsi_user:change_me@localhost:5432/gipsi" -c "SELECT 1;"
```

Listing 6.3: Preparación recomendada de PostgreSQL

Al arrancar, Flyway valida y aplica automáticamente las migraciones pendientes. Esto significa que la estructura de la base de datos queda sincronizada con la versión desplegada de la API. Si una migración falla, la aplicación no debe considerarse lista para recibir tráfico hasta revisar el error y recuperar la base de datos desde una copia consistente si fuera necesario.

6.3. Configuración del Frontend

6.3.1. Bootstrap del Monorepo

```
# 1. Instalar Melos globalmente
dart pub global activate melos

# 2. Instalar dependencias de todos los packages
melos bootstrap

# 3. Generar carpetas de plataforma (solo primera vez)
cd apps/gipsi
flutter create --project-name gipsi \
               --org com.gipsi \
               --platforms=android,ios .

# 4. Lanzar la app (emulador Android apunta a 10.0.2.2:8080)
flutter run

# Para iOS Simulator
flutter run --dart-define=API_URL=http://localhost:8080

# Para dispositivo físico en red local
flutter run --dart-define=API_URL=http://192.168.1.10:8080

# Para producción
flutter run --dart-define=API_URL=https://api.gipsi.com
```

Listing 6.4: Proceso de setup del frontend

La variable `API_URL` se inyecta en tiempo de compilación mediante `-dart-define`. Esto implica que cada build queda ligado al entorno para el que se generó: un APK

compilado contra `http://10.0.2.2:8080` servirá para emulador Android, pero no para un dispositivo físico ni para producción. En iOS Simulator puede usarse `localhost`, mientras que en un móvil real se necesita una IP accesible desde la misma red o una URL pública con HTTPS.

Esta separación también facilita tener builds distintos para desarrollo, preproducción y producción. Lo importante es que la URL configurada en `API_URL` coincida con los orígenes permitidos en `CORS_ALLOWED_ORIGINS` cuando corresponda, y que los dispositivos cliente puedan resolver el dominio de la API.

6.3.2. Compilación para Distribución

```
# Android - APK de debug
flutter build apk --debug

# Android - APK de release (requiere keystore configurado)
flutter build apk --release \
  --dart-define=API_URL=https://api.gipsi.com

# Android - App Bundle para Google Play
flutter build appbundle --release \
  --dart-define=API_URL=https://api.gipsi.com

# iOS - Archivo para App Store (requiere cuenta Apple Developer)
flutter build ios --release \
  --dart-define=API_URL=https://api.gipsi.com
```

Listing 6.5: Compilación para Android e iOS

Los builds de depuración son adecuados para validar la integración durante el desarrollo, pero no para distribuir la aplicación. En Android, la versión `release` debe firmarse con un keystore propio y, si se publica en Google Play, normalmente se genera un `appbundle`. En iOS, la generación del archivo de distribución requiere certificados, perfiles de provisión y una cuenta Apple Developer. En ambos casos, la URL de la API debe quedar fijada en el comando de compilación para evitar publicar una app apuntando a un backend local.

6.4. Despliegue con Docker (Recomendado para Producción)

Aunque no existe un `Dockerfile` en el repositorio actual, la configuración recomendada para producción es la siguiente:

La composición propuesta agrupa dos responsabilidades: PostgreSQL como servicio persistente y la API como servicio de aplicación. La base de datos conserva sus datos en el volumen `pgdata`, mientras que las imágenes gestionadas por la API se guardan en el volumen `storage`. Esta separación permite actualizar la imagen de la API sin perder información de negocio ni ficheros subidos por los usuarios.

```
version: '3.9'
services:
  postgres:
    image: postgres:16-alpine
```

```

environment:
  POSTGRES_DB: gipsi
  POSTGRES_USER: gipsi_user
  POSTGRES_PASSWORD: ${DB_PASSWORD}
volumes:
  - pgdata:/var/lib/postgresql/data
healthcheck:
  test: ["CMD", "pg_isready", "-U", "gipsi_user"]
  interval: 10s

api:
  image: gipsi-api:latest
  depends_on:
    postgres:
      condition: service_healthy
  environment:
    DB_URL: jdbc:postgresql://postgres:5432/gipsi
    DB_USER: gipsi_user
    DB_PASSWORD: ${DB_PASSWORD}
    JWT_SECRET: ${JWT_SECRET}
    REFRESH_EXPIRATION_DAYS: 30
    CORS_ALLOWED_ORIGINS: https://app.gipsi.com
    GIPSI_BASE_URL: https://api.gipsi.com
    STORAGE_TYPE: local
    STORAGE_PATH: /var/gipsi/storage
    FIREBASE_CREDENTIALS_PATH: ${FIREBASE_CREDENTIALS_PATH:-}
    SPRING_PROFILES_ACTIVE: prod
  ports:
    - "8080:8080"
  volumes:
    - storage:/var/gipsi/storage

volumes:
  pgdata:
  storage:

```

Listing 6.6: docker-compose.yml para producción

6.4.1. Archivo de Variables para Docker

En un despliegue real, las variables no deberían escribirse directamente en el fichero de composición de Docker. Es preferible mantener un fichero `.env` fuera del control de versiones o utilizar el gestor de secretos de la plataforma donde se ejecute el servicio.

```

DB_PASSWORD=replace_with_a_long_random_password
JWT_SECRET=replace_with_output_of_openssl_rand_base64_64
FIREBASE_CREDENTIALS_PATH=

```

Listing 6.7: Ejemplo de fichero `.env` de producción

El secreto JWT puede generarse con `openssl rand -base64 64`. Este valor debe conservarse entre reinicios, porque cambiarlo invalida los tokens firmados previamente y obliga a los usuarios a autenticarse de nuevo. La contraseña de PostgreSQL debe rotarse con cuidado, actualizando primero el servidor de base de datos y después las variables de la API.

6.4.2. Secuencia de Puesta en Producción

Una vez preparado el fichero de entorno y publicada la imagen `gipsi-api:latest`, la puesta en marcha puede realizarse con la siguiente secuencia:

```
# 1. Validar que existen las variables requeridas
docker compose --env-file .env config

# 2. Arrancar servicios en segundo plano
docker compose --env-file .env up -d

# 3. Revisar logs de la API y migraciones Flyway
docker compose logs -f api

# 4. Comprobar que la documentación OpenAPI responde
curl -f http://localhost:8080/swagger-ui.html
```

Listing 6.8: Secuencia básica de despliegue Docker

En un servidor público, la última comprobación debería hacerse contra la URL HTTPS expuesta por el proxy inverso. Si la API responde localmente pero no desde el dominio público, el problema suele estar en la configuración del proxy, del certificado TLS, del firewall o de los registros DNS, no necesariamente en la aplicación Spring Boot.

6.4.3. Verificaciones Posteriores y Mantenimiento

Después del despliegue no basta con que el contenedor aparezca como levantado. La aplicación debe validarse con operaciones representativas del sistema:

- Acceder a `/swagger-ui.html` o `/v3/api-docs` para confirmar que la API está disponible.
- Registrar o autenticar un usuario de prueba y comprobar la emisión de access token y refresh token.
- Crear una entidad que suba imagen, verificando que el volumen `storage` persiste el fichero.
- Reiniciar la API y confirmar que los datos siguen presentes en PostgreSQL y en el almacenamiento.
- Si Firebase está configurado, registrar un token de dispositivo y probar el envío de una notificación.

También deben planificarse copias de seguridad periódicas de PostgreSQL y del volumen de imágenes. Una copia de la base de datos puede generarse con:

```
docker compose exec postgres pg_dump -U gipsi_user gipsi > backup_gipsi.sql
```

Listing 6.9: Copia de seguridad básica de PostgreSQL

Para actualizar la API, el procedimiento recomendado es publicar una nueva imagen, arrancarla con la misma configuración externa y revisar los logs de Flyway. Si la actualización incluye migraciones de base de datos, conviene realizar previamente una copia de seguridad y probar el proceso en un entorno de preproducción con datos equivalentes.

Capítulo 7

Conclusiones y Posibles Mejoras

7.1. Conclusiones

El desarrollo de Gipsi ha permitido construir una plataforma funcional de gestión de citas para el sector de la estética y el bienestar, demostrando la viabilidad técnica de la combinación Spring Boot + Flutter para aplicaciones de gestión con requisitos de seguridad y disponibilidad real.

7.1.1. Logros Técnicos Destacados

Arquitectura de seguridad robusta. La implementación del sistema de autenticación JWT con rotación de refresh tokens sigue las recomendaciones OWASP y proporciona protección real ante el robo de tokens. La revocación en cadena cuando se detecta uso de un token ya rotado es una característica de nivel profesional.

Monorepo Flutter con Melos. La organización del frontend en tres packages interdependientes (`gipsi`, `gipsi_api`, `gipsi_shared_ui`) establece una base sólida para la escalabilidad del proyecto. La separación entre lógica de red, sistema de diseño y app ejecutable facilita el testing independiente y la reutilización futura.

Cálculo de disponibilidad. El motor de disponibilidad horaria considera múltiples variables simultáneas (horario del negocio, reservas existentes, días libres del trabajador, política de horas extra) con una respuesta bien tipada que indica la causa específica de cada indisponibilidad, lo que permite una UX informativa para el cliente.

Abstracción del almacenamiento. La interfaz `FileStorage` con implementación local y esqueleto S3 es un ejemplo real del principio de inversión de dependencias aplicado con un propósito práctico: la migración a almacenamiento cloud no requerirá modificar ningún servicio de negocio.

7.1.2. Lecciones Aprendidas

- La gestión de estado en Flutter con Riverpod es significativamente más manejable que con soluciones más rudimentarias (`setState` global), especialmente cuando el estado de autenticación debe afectar a múltiples partes de la UI de forma reactiva.

- Las migraciones de base de datos con Flyway son imprescindibles desde el primer día de desarrollo, no como añadido posterior, porque el historial de migraciones es en sí mismo documentación del modelo de datos.
- La documentación automática con Springdoc OpenAPI no es solo un lujo: acelera enormemente el desarrollo del frontend al proporcionar un contrato ejecutable y actualizado de la API.

7.2. Posibles Mejoras

7.2.1. Mejoras a Corto Plazo (Bloques Pendientes)

1. **Flujo de reserva completo (Bloque 3):** Implementar los cuatro pasos de reserva en el frontend: selección de servicio, selección de trabajador o «cualquier disponible», selección de fecha y hora desde el calendario de disponibilidad, y pantalla de confirmación.
2. **Gestión de citas propias (Bloque 5):** La pestaña «Mis citas» necesita listar las reservas del cliente, permitir su cancelación dentro del plazo permitido y habilitar el flujo de valoración post-servicio.
3. **Tests automatizados con Testcontainers:** El backend carece de pruebas de integración contra una base de datos real. Testcontainers permite levantar un contenedor PostgreSQL en tiempo de prueba, eliminando la necesidad de un servidor externo y haciendo las pruebas reproducibles en cualquier entorno CI.
4. **Búsqueda con filtros y favoritos:** La pestaña «Buscar» está preparada en navegación pero sin implementación. Los favoritos requieren el endpoint `PUT /customers/me/favorites/businesses/{id}` en el backend.

7.2.2. Mejoras Arquitectónicas

1. **Migración a almacenamiento S3/R2:** Sustituir `LocalFileStorage` por `S3FileStorage` usando el SDK de AWS o la API compatible S3 de Cloudflare R2, que es más económica para proyectos en sus fases iniciales. El código ya dispone del esqueleto necesario.
2. **Caché de queries frecuentes:** Endpoints como `GET /categories` o `GET /businesses` con filtros comunes pueden beneficiarse de una caché en memoria (Caffeine integrado en Spring) o distribuida (Redis) para reducir carga en la base de datos.
3. **Paginación por cursor:** La paginación actual usa `page/size` (offset-based), que tiene problemas de consistencia cuando se insertan o eliminan registros entre páginas. Una paginación por cursor (keyset pagination) es más eficiente y estable para feeds en tiempo real.
4. **App de negocio independiente:** La arquitectura del monorepo facilita crear `apps/gipsi_business` que comparta `gipsi_api` y `gipsi_shared_ui` pero ofrezca una experiencia específica para propietarios (gestión de agenda, perfil de negocio, estadísticas).
5. **Pipeline CI/CD:** Configurar GitHub Actions con: ejecución de tests del backend con Testcontainers, análisis estático con `flutter analyze` y `dart analyze`, build

del APK de Android en cada push a `main`, y despliegue automático al servidor de producción.

7.2.3. Mejoras de Seguridad

1. **HTTPS obligatorio:** En producción, toda comunicación debe ser HTTPS. El backend no gestiona certificados (esa responsabilidad recae en un proxy inverso como Nginx o Caddy), pero debe configurarse para rechazar peticiones no seguras.
2. **Validación de tipos MIME en upload:** Al subir imágenes, verificar que el tipo MIME real del fichero (magic bytes) coincide con la extensión declarada, para prevenir upload de ficheros maliciosos disfrazados de imágenes.
3. **Auditoría de accesos:** Registrar en base de datos los intentos de autenticación fallidos, rotaciones de tokens y acciones administrativas para facilitar la detección de incidentes.

Bibliografía

- [1] Pivotal Software. *Spring Boot Reference Documentation, version 3.4*. 2024. <https://docs.spring.io/spring-boot/docs/3.4.x/reference/html/>
- [2] Spring Security Team. *Spring Security Reference, version 6*. 2024. <https://docs.spring.io/spring-security/reference/>
- [3] Stormpath / io.jsonwebtoken. *JJWT — Java JWT library, version 0.12*. 2024. <https://github.com/jwtkt/jjwt>
- [4] M. Jones, J. Bradley, N. Sakimura. *JSON Web Token (JWT)*. RFC 7519. Internet Engineering Task Force, mayo 2015. <https://www.rfc-editor.org/rfc/rfc7519>
- [5] OWASP Foundation. *JSON Web Token Cheat Sheet for Java*. 2024. https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html
- [6] Google LLC. *Flutter documentation*. 2024. <https://docs.flutter.dev/>
- [7] Remi Rousselet. *Riverpod documentation*. 2024. <https://riverpod.dev/>
- [8] Flutter team. *go_router package documentation*. 2024. https://pub.dev/packages/go_router
- [9] Flute DEV. *Dio — A powerful HTTP networking package for Dart/Flutter*. 2024. <https://pub.dev/packages/dio>
- [10] Invertase. *Melos — A tool for managing Dart/Flutter monorepos*. 2024. <https://melos.invertase.dev/>
- [11] Redgate. *Flyway — Database schema migration made easy*. 2024. <https://flywaydb.org/documentation/>
- [12] Vladimir Bukhtoyarov. *Bucket4j — Rate limiting library for Java*. 2024. <https://github.com/bucket4j/bucket4j>
- [13] Google LLC. *Firebase Cloud Messaging documentation*. 2024. <https://firebase.google.com/docs/cloud-messaging>
- [14] Springdoc contributors. *springdoc-openapi — Library for Spring Boot projects*. 2024. <https://springdoc.org/>
- [15] The PostgreSQL Global Development Group. *PostgreSQL 16 Documentation*. 2024. <https://www.postgresql.org/docs/16/>
- [16] AtomicJar. *Testcontainers for Java*. 2024. <https://java.testcontainers.org/>
- [17] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation, University of California, Irvine, 2000. <https://>

[//ics.uci.edu/~fielding/pubs/dissertation/top.htm](https://ics.uci.edu/~fielding/pubs/dissertation/top.htm)

- [18] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [19] Google LLC. *Dart language tour*. 2024. <https://dart.dev/language>

Apéndice A

Configuración Completa del pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.4.4</version>
  </parent>
  <groupId>com.gipsi</groupId>
  <artifactId>gipsi-backend</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <properties>
    <java.version>21</java.version>
    <jjwt.version>0.12.6</jjwt.version>
    <springdoc.version>2.8.5</springdoc.version>
  </properties>
  <dependencies>
    <!-- Spring Boot Starters -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-security</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <!-- Base de datos -->
    <dependency>
```

```

    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-core</artifactId>
</dependency>
<dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-database-postgresql</artifactId>
</dependency>
<!-- JWT -->
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>${jjwt.version}</version>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>${jjwt.version}</version>
    <scope>runtime</scope>
</dependency>
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>${jjwt.version}</version>
    <scope>runtime</scope>
</dependency>
<!-- OpenAPI / Swagger -->
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>${springdoc.version}</version>
</dependency>
<!-- Firebase Admin SDK -->
<dependency>
    <groupId>com.google.firebase</groupId>
    <artifactId>firebase-admin</artifactId>
    <version>9.4.1</version>
</dependency>
<!-- Rate Limiting -->
<dependency>
    <groupId>com.bucket4j</groupId>
    <artifactId>bucket4j-core</artifactId>
    <version>8.10.1</version>
</dependency>
<!-- Testing -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.springframework.security</groupId>
    <artifactId>spring-security-test</artifactId>
    <scope>test</scope>

```

```
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

Listing A.1: pom.xml del proyecto Gipsi-API

Apéndice B

Configuración del Monorepo (melos.yaml)

```
name: gipsi_monorepo
repository: https://github.com/gipsi/gipsi-frontend

packages:
  - apps/*
  - packages/*

command:
  clean:
    hooks:
      post: melos exec -- "flutter clean"
  bootstrap:
    runPubGetInParallel: false

scripts:
  analyze:
    run: melos exec -- "flutter analyze"
    description: Ejecuta flutter analyze en todos los paquetes

  format:
    run: melos exec -- "dart format ."
    description: Formatea todo el código

  test:
    run: melos exec --dir-exists="test" -- "flutter test"
    description: Ejecuta los tests

  gen:
    run: >
      melos exec --depends-on="build_runner" --
        "dart run build_runner build --delete-conflicting-outputs"
    description: Regenera los archivos freezed/json_serializable
```

Listing B.1: melos.yaml del monorepo Flutter

Apéndice C

Usuarios de Prueba (DataInitializer)

La clase `DataInitializer` se activa con el perfil `dev` y crea los siguientes usuarios de prueba para facilitar el desarrollo y las demos:

Tabla C.1: Usuarios de prueba creados por `DataInitializer`

Nombre de usuario	Rol	Contraseña
<code>laurasanchez</code>	Cliente	<code>password123</code>
<code>migueltorres</code>	Cliente	<code>password123</code>
<code>sofiaruiz</code>	Cliente	<code>password123</code>
<code>peluqueriastyle</code>	Negocio	<code>password123</code>
<code>elmaestrobarbero</code>	Negocio	<code>password123</code>
<code>nailartstudio</code>	Negocio	<code>password123</code>
<code>esteticaandaluza</code>	Negocio	<code>password123</code>
<code>spabienestar</code>	Negocio	<code>password123</code>
<code>mariagarcia</code>	Trabajador	<code>password123</code>
<code>carloslopez</code>	Trabajador	<code>password123</code>
<code>anamartinez</code>	Trabajador	<code>password123</code>

Advertencia

Los usuarios de prueba **nunca** deben estar presentes en entornos de producción. El perfil `dev` (y por tanto el `DataInitializer`) debe estar desactivado mediante la variable `SPRING_PROFILES_ACTIVE=prod`.

Apéndice D

Formato de Respuesta de Disponibilidad

El endpoint GET `/services/{id}/availability?date=YYYY-MM-DD&workerId={id}` devuelve el siguiente formato JSON:

```
{
  "serviceId": 5,
  "workerId": 12,
  "date": "2026-07-15",
  "slotMinutes": 15,
  "serviceDuration": 45,
  "slots": [
    {
      "start": "09:00",
      "end": "09:45",
      "available": true,
      "reason": null,
      "availableWorkerIds": null
    },
    {
      "start": "09:15",
      "end": "10:00",
      "available": true,
      "reason": null,
      "availableWorkerIds": null
    },
    {
      "start": "09:30",
      "end": "10:15",
      "available": false,
      "reason": "BOOKED",
      "availableWorkerIds": null
    },
    {
      "start": "14:00",
      "end": "14:45",
      "available": false,
      "reason": "OUT_OF_SCHEDULE",
      "availableWorkerIds": null
    }
  ]
}
```

```
}
```

Listing D.1: Ejemplo de respuesta del endpoint de disponibilidad

Razones de indisponibilidad posibles:

BOOKED Existe una reserva confirmada o pendiente que se solapa con este slot.

OUT_OF_SCHEDULE El slot cae fuera del horario de apertura del negocio.

TIME_OFF El trabajador tiene registrado un día libre o ausencia.

OVERTIME_NOT_ALLOWED El servicio terminaría después del cierre y no se permiten horas extra.

WORKER_UNAVAILABLE Ningún trabajador habilitado está libre en este slot (modo multi-worker).

Apéndice E

Glosario de Términos

E.1. Glosario de términos técnicos

Access Token Token JWT de corta duración, de 30 minutos en Gipsi, que se envía en la cabecera **Authorization** de cada petición autenticada. Sirve para identificar al usuario que realiza la petición y comprobar sus permisos sin consultar la sesión en cada solicitud.

API REST Interfaz de comunicación entre el frontend móvil y el backend. Utiliza HTTP como protocolo de transporte y organiza las operaciones mediante recursos, métodos y respuestas estructuradas, normalmente en formato JSON.

Backend Parte del sistema ejecutada en el servidor. En Gipsi se encarga de gestionar la lógica de negocio, la seguridad, la base de datos, las reservas, los usuarios, las valoraciones, las notificaciones y la exposición de endpoints a través de la API REST.

Bucket4j Librería Java utilizada para aplicar mecanismos de limitación de peticiones. Permite controlar cuántas solicitudes puede realizar un cliente en un periodo determinado, ayudando a proteger la API frente a abuso, errores o ataques de fuerza bruta.

Dart Lenguaje de programación fuertemente tipado, orientado a objetos y desarrollado por Google. Es el lenguaje utilizado por Flutter para construir aplicaciones móviles, web y de escritorio desde una misma base de código.

Dio Cliente HTTP para Dart y Flutter. En Gipsi se utiliza para realizar peticiones a la API, gestionar cabeceras, procesar respuestas y configurar interceptores, por ejemplo para añadir automáticamente el access token o refrescar credenciales cuando sea necesario.

DTO Data Transfer Object. Objeto utilizado para transferir datos entre capas o entre cliente y servidor. Permite separar el modelo interno de base de datos de la información que realmente se expone o recibe a través de la API.

Firestore Cloud Messaging Servicio de Google utilizado para enviar notificaciones push a dispositivos móviles. En Gipsi permite avisar a los usuarios de eventos relevantes, como confirmaciones, cambios o recordatorios relacionados con sus citas.

Flutter Framework de desarrollo multiplataforma creado por Google. Permite construir interfaces móviles nativas para Android e iOS utilizando Dart y una arquitectura basada en widgets.

Flyway Herramienta de migración de esquema de base de datos. Aplica scripts SQL versionados en orden, permitiendo que la estructura de PostgreSQL evolucione de forma controlada entre entornos de desarrollo, pruebas y producción.

Frontend Parte visible de la aplicación con la que interactúa el usuario. En Gipsi corresponde a la aplicación móvil desarrollada con Flutter, encargada de mostrar pantallas, formularios, navegación, estados de carga y respuestas procedentes del backend.

go_router Librería de navegación declarativa para Flutter. Permite definir las rutas de la aplicación, gestionar parámetros, redirecciones, rutas protegidas y estructuras de navegación como pestañas o layouts persistentes.

Hibernate Implementación de JPA utilizada habitualmente por Spring Boot. Se encarga de traducir entidades Java a tablas relacionales y de generar las operaciones SQL necesarias para consultar, insertar, actualizar o eliminar datos.

JPA Java Persistence API. Especificación estándar de Java para mapear objetos Java a tablas relacionales. Permite trabajar con entidades, relaciones y consultas sin escribir SQL manual en la mayoría de operaciones comunes.

JSON JavaScript Object Notation. Formato ligero de intercambio de datos utilizado en la comunicación entre frontend y backend. Es legible para humanos y fácil de procesar por aplicaciones.

JSON Web Token Formato estándar de token firmado que permite transportar información de identidad y autorización. En Gipsi se utiliza para autenticar peticiones sin mantener una sesión tradicional en memoria del servidor.

Melos Herramienta de gestión de monorepos Dart y Flutter. Permite administrar varios paquetes dentro de un mismo repositorio y ejecutar comandos comunes como `bootstrap`, `test` o `analyze` sobre todos ellos.

Monorepo Estrategia de organización del código donde varios módulos, aplicaciones o paquetes conviven dentro de un único repositorio. En Gipsi facilita compartir código entre paquetes como la aplicación principal, la capa de datos o el sistema de diseño.

OpenAPI Especificación estándar para describir APIs REST. Define endpoints, métodos HTTP, parámetros, cuerpos de petición, respuestas y modelos de datos, facilitando la documentación y la integración con otros sistemas.

PostgreSQL Sistema gestor de base de datos relacional utilizado por Gipsi. Almacena la información persistente del sistema, como usuarios, negocios, servicios, citas, valoraciones y tokens.

Provider (Riverpod) Unidad básica de Riverpod que encapsula un estado, una dependencia o una función. Los widgets pueden observar providers para reconstruirse automáticamente cuando el estado cambia.

Rate Limiting Técnica de control de tráfico que limita el número de peticiones que un cliente puede realizar en un intervalo de tiempo. Se utiliza para proteger la

API frente a uso abusivo, errores de integración o ataques automatizados.

Refresh Token Token opaco de larga duración, de 30 días en Gipsi, almacenado en base de datos y usado exclusivamente para obtener un nuevo par de tokens. Se rota en cada uso para reducir el riesgo de reutilización indebida si el token anterior queda comprometido.

Repository Patrón de arquitectura que encapsula el acceso a datos. En el frontend permite que la interfaz no dependa directamente de Dio ni de los detalles de la API, sino de una capa intermedia que ofrece métodos de negocio más claros.

REST Representational State Transfer. Estilo arquitectónico para sistemas distribuidos que define una forma de comunicación cliente-servidor basada en recursos, métodos HTTP y respuestas sin estado.

Riverpod Solución de gestión de estado para Flutter. Permite manejar datos reactivos, dependencias y lógica compartida entre pantallas de forma segura, testeable y desacoplada del árbol de widgets.

S3 Sistema de almacenamiento de objetos popularizado por Amazon Simple Storage Service. Se utiliza para guardar archivos, imágenes u otros recursos binarios fuera de la base de datos. En Gipsi existe una capa de abstracción preparada para migrar el almacenamiento de imágenes a S3 o a servicios compatibles en el futuro.

ShellRoute Concepto de `go_router` que permite mantener una estructura visual persistente, como un **Scaffold** con barra de navegación inferior, mientras se sustituye únicamente el contenido correspondiente a la ruta activa.

JOINED Inheritance Estrategia de herencia en JPA donde la entidad base y cada subtipo se almacenan en tablas separadas, enlazadas por la misma clave primaria. Permite compartir campos comunes en una tabla base sin mezclar en una sola tabla todos los atributos específicos.

Slot Franja horaria calculada por el motor de disponibilidad. En Gipsi la granularidad por defecto es de 15 minutos, lo que permite dividir la agenda en bloques reservables y comprobar qué huecos están libres según servicios, trabajadores y reservas existentes.

Spring Boot Framework Java que simplifica la creación de aplicaciones backend. Proporciona configuración automática, integración con seguridad, persistencia, validación, documentación y despliegue de APIs REST.

Springdoc OpenAPI Librería que genera automáticamente documentación OpenAPI a partir de los controladores y modelos definidos en Spring Boot. Permite disponer de una documentación técnica actualizada de la API sin mantenerla manualmente.

Token Bucket Algoritmo de control de flujo utilizado por herramientas como Bucket4j para implementar rate limiting con soporte de ráfagas. Funciona como un depósito de tokens que se consume con cada petición y se rellena progresivamente con el tiempo.

Widget Unidad básica de construcción de interfaces en Flutter. Todo elemento visual, desde un texto hasta una pantalla completa, se representa mediante widgets que pueden componerse entre sí.

Android App Bundle Formato de distribución recomendado para publicar aplica-

ciones Android en Google Play. A partir de este paquete, la tienda genera APKs optimizados para cada dispositivo, reduciendo el tamaño de descarga frente a un APK universal.

APK Android Package Kit. Archivo instalable de una aplicación Android. En Gipsi se usa principalmente para compilaciones de depuración o pruebas manuales antes de preparar una versión de distribución.

@Async Anotación de Spring que permite ejecutar un método en segundo plano, fuera del hilo principal que atiende la petición HTTP. En Gipsi se utiliza para enviar notificaciones push sin bloquear la respuesta al usuario.

@EventListener Anotación de Spring que permite reaccionar a eventos internos de la aplicación. En Gipsi se aplica para desacoplar acciones como la creación de una reserva del envío posterior de notificaciones.

@Scheduled Anotación de Spring usada para ejecutar tareas programadas según una expresión temporal. En Gipsi permite automatizar procesos como la cancelación de reservas pendientes vencidas o el envío diario de recordatorios.

BCrypt Algoritmo de hash de contraseñas diseñado para ser resistente a ataques de fuerza bruta. Spring Security lo utiliza mediante `BCryptPasswordEncoder` para almacenar contraseñas sin guardar el texto original.

Bearer Token Esquema de autorización HTTP en el que el cliente envía un token dentro de la cabecera `Authorization`. En Gipsi se usa con la forma `Authorization: Bearer <token>` para autenticar peticiones protegidas.

Claim Afirmación incluida en el payload de un JWT, como el identificador del usuario, sus roles o la fecha de expiración. Estas claims permiten al backend reconstruir la identidad del usuario en APIs stateless.

CORS Cross-Origin Resource Sharing. Mecanismo de seguridad del navegador que controla que orígenes pueden realizar peticiones a una API. En Gipsi se configura mediante variables de entorno para permitir solo clientes autorizados en producción.

Controller Capa del backend encargada de recibir peticiones HTTP, validar la entrada básica y delegar la operación en los servicios. En la arquitectura de Gipsi no contiene lógica de negocio compleja.

Cron Sintaxis para definir calendarios de ejecución de tareas programadas. En Gipsi aparece en los jobs de Spring que se ejecutan cada hora o cada mañana.

CSRF Cross-Site Request Forgery. Tipo de ataque que intenta forzar acciones autenticadas desde otro sitio web. En APIs REST stateless como Gipsi se deshabilita porque no se usan sesiones basadas en cookies del servidor.

DataInitializer Componente de arranque utilizado en el perfil de desarrollo para cargar usuarios, negocios, servicios y datos de prueba. Facilita probar la aplicación sin introducir manualmente toda la información inicial.

Device Token Identificador emitido por Firebase Cloud Messaging para representar un dispositivo concreto. Gipsi lo asocia al usuario para saber a que terminales enviar notificaciones push.

- Docker Compose** Herramienta para definir y levantar varios servicios mediante un archivo YAML. En el despliegue propuesto de Gipsi coordina la API, PostgreSQL y los volúmenes persistentes.
- Endpoint** Ruta concreta de una API que expone una operación o recurso, por ejemplo `POST /auth/login` o `GET /businesses`. Cada endpoint define un contrato de entrada, autenticación y respuesta.
- Entidad** Clase Java mapeada a una tabla de base de datos mediante JPA. Representa objetos persistentes del dominio, como usuarios, negocios, reservas, servicios o valoraciones.
- Estado vacío** Estado de interfaz mostrado cuando una consulta no devuelve resultados. En Gipsi se usa, por ejemplo, en la pantalla de exploración para explicar que no hay negocios que coincidan con los filtros aplicados.
- FileStorage** Interfaz interna que abstrae el almacenamiento de archivos. Permite que la lógica de negocio no dependa de si las imágenes se guardan localmente o en un servicio externo compatible con S3.
- Firestore Admin SDK** SDK de servidor que permite a Spring Boot comunicarse con Firestore. En Gipsi se usa para enviar mensajes FCM desde el backend a los dispositivos registrados.
- flutter_secure_storage** Paquete de Flutter usado para guardar información sensible en el almacenamiento seguro del sistema operativo. En Android utiliza Keystore y en iOS Keychain para proteger los tokens de sesión.
- GlobalExceptionHandler** Componente centralizado para transformar excepciones del backend en respuestas HTTP consistentes. Evita repetir manejo de errores en cada controlador y mejora la claridad de la API.
- Granularidad** Tamaño mínimo de división usado por un algoritmo. En el motor de disponibilidad de Gipsi la granularidad de 15 minutos determina cada cuanto se generan slots reservables.
- Healthcheck** Comprobación automática que indica si un servicio está listo para recibir tráfico. En el despliegue con Docker se usa para esperar a que PostgreSQL esté disponible antes de arrancar la API.
- HTTP 401 Unauthorized** Código de respuesta que indica que la petición no está autenticada correctamente. En Gipsi puede activar el interceptor de Dio para intentar renovar el access token.
- HTTP 429 Too Many Requests** Código de respuesta que indica que el cliente ha superado el límite de peticiones. Es la respuesta esperada cuando el rate limiting rechaza una solicitud por falta de tokens en el bucket.
- Interceptor** Componente que intercepta peticiones o respuestas HTTP para añadir comportamiento transversal. En Gipsi, el interceptor de Dio incorpora tokens, detecta errores 401 y reintenta peticiones tras refrescar credenciales.
- JUnit 5** Framework de pruebas unitarias para Java. En Gipsi se utiliza para verificar servicios del backend, especialmente lógica de autenticación, reservas y disponibilidad.

Keychain Almacén seguro de credenciales de iOS. La aplicación Flutter lo utiliza, a través de `flutter_secure_storage`, para proteger tokens y datos sensibles del usuario.

Keystore Sistema seguro de Android para proteger claves y credenciales. En Gipsi se usa indirectamente mediante `flutter_secure_storage` para evitar que los tokens queden expuestos como texto plano.

LocalFileStorage Implementación local de la interfaz `FileStorage`. Guarda archivos en disco y sirve como solución inicial antes de migrar a almacenamiento cloud.

Maven Herramienta de gestión y construcción de proyectos Java. En Gipsi compila el backend, descarga dependencias, ejecuta pruebas y genera el archivo JAR desplegable.

Migración Script versionado que modifica el esquema de base de datos de forma controlada. Flyway aplica estas migraciones en orden para mantener sincronizados los entornos.

OWASP Open Worldwide Application Security Project. Comunidad y conjunto de guías de buenas prácticas de seguridad. La rotación de refresh tokens de Gipsi sigue recomendaciones alineadas con estas guías.

Payload Parte de un JWT que contiene las claims del token. Aunque está codificada, no debe tratarse como información secreta; la seguridad del token depende de su firma y de transportarlo por canales seguros.

Pull-to-refresh Patrón de interfaz en aplicaciones móviles que permite recargar contenido arrastrando la lista hacia abajo. En Gipsi se aplica en la pantalla de exploración.

Repositorio de datos Capa de Spring Data JPA que ofrece operaciones de consulta y persistencia sobre entidades. Reduce la necesidad de escribir SQL manual para operaciones comunes.

Rotación de Refresh Tokens Estrategia de seguridad en la que cada uso del refresh token emite uno nuevo e invalida el anterior. Si un token ya rotado se reutiliza, Gipsi puede detectar el abuso y revocar la cadena de tokens.

Service Capa de la arquitectura backend que concentra la lógica de negocio. En Gipsi incluye clases como `BookingService`, `AuthService` o `AvailabilityService`.

Signature Firma criptográfica de un JWT calculada a partir del header, el payload y el secreto del servidor. Permite detectar si alguien ha modificado el token.

Skeleton loading Patrón visual que muestra estructuras de carga con forma similar al contenido final. En la exploración de Gipsi evita una pantalla vacía mientras se recupera la primera página de resultados.

SliverAppBar Componente de Flutter usado dentro de un `CustomScrollView` para crear barras superiores flexibles y colapsables. En Gipsi se utiliza en el detalle de negocio para mostrar una portada que cambia al hacer scroll.

Spring Security Módulo de Spring encargado de autenticación, autorización y filtros de seguridad. En Gipsi valida JWT, aplica reglas por rol y protege endpoints privados.

Stateless Propiedad de una API que no guarda sesión del usuario en el servidor entre peticiones. Cada solicitud debe aportar sus credenciales, normalmente mediante un token.

Swagger UI Interfaz web generada a partir de OpenAPI para explorar y probar endpoints de la API. En Gipsi se expone durante el desarrollo para facilitar validación e integración.

Transactional Concepto asociado a operaciones de base de datos que deben ejecutarse como una unidad: si una parte falla, se deshacen los cambios. En Spring se aplica mediante `@Transactional`.

Variable de entorno Valor externo usado para configurar la aplicación sin escribir credenciales ni ajustes específicos en el código fuente. Gipsi las usa para base de datos, JWT, CORS, Firebase y rutas de almacenamiento.

Volumen Docker Mecanismo de persistencia usado por Docker para conservar datos fuera del ciclo de vida de los contenedores. En el despliegue de Gipsi se usa para PostgreSQL y almacenamiento de archivos.